

Streszczenie

Przedmiotem dokumentacji jest dwukołowy robot mobilny utrzymujący pionową równowagę na zasadzie odwróconego wahadła. Robot jest układem z natury niestabilnym: jego środek masy znajduje się powyżej osi kół, dlatego utrzymanie pozycji pionowej wymaga ciągłej, aktywnej regulacji.

Pomiar pochylenia realizowany jest czujnikiem inercyjnym MPU-6050 (akcelerometr + żyroskop), którego sygnały łączone są filtrem komplementarnym. Na podstawie estymowanego kąta regulator PID wyznacza zadaną prędkość kół. Napęd stanowią dwa silniki krokowe NEMA 17 sterowane modułami TMC2226 w trybie STEP/DIR; impulsy krokowe generowane są sprzętowo przez przerwanie 16-bitowego licznika Timer1 mikrokontrolera ATmega328P. Pętla regulacji pracuje z częstotliwością 250 Hz.

Dokument obejmuje podstawy teoretyczne, architekturę sprzętową i programową, wykaz komponentów (BOM), opis konstrukcji mechanicznej i elektroniki, pełny listing oprogramowania sterującego, procedurę uruchomienia oraz strojenia regulatora, a także wyniki testów i kierunki dalszego rozwoju.

Słowa kluczowe: robot samobalansujący, odwrócone wahadło, regulator PID, filtr komplementarny, MPU-6050, silnik krokowy, TMC2226, ATmega328P.

Wykaz ważniejszych oznaczeń i skrótów

Symbol / skrót	Znaczenie
θ	estymowany kąt pochylenia korpusu od pionu [°]
θ_{acc}	kąt wyznaczony z akcelerometru
ω	prędkość kątowna pochylenia (żyroskop) [°/s]
α	współczynnik filtra komplementarnego (0,98)
e	uchyb regulacji, $e = \theta_{zad} - \theta$
K_p, K_i, K_d	nastawy regulatora PID (wzmocnienia)
Δt	krok czasowy pętli regulacji (≈ 4 ms)
u	sygnał sterujący (zadana prędkość kół)
IMU	jednostka inercyjna (Inertial Measurement Unit)
PID	regulator proporcjonalno-całkująco-różniczkujący
PWM	modulacja szerokości impulsu
ISR	procedura obsługi przerwania (Interrupt Service Routine)
CTC	tryb licznika Clear Timer on Compare
DLPF	cyfrowy filtr dolnoprzepustowy (w MPU-6050)
LSB	najmniej znaczący bit (rozdzielczość przetwornika)
BOM	wykaz materiałowy (Bill of Materials)
COM / CoG	środek masy / środek ciężkości

1. Wstęp

1.1. Cel i zakres projektu

Celem projektu była budowa i uruchomienie dwukołowego robota mobilnego, który samodzielnie utrzymuje równowagę w pionie, balansując na dwóch współosiowych kołach. Zadanie należy do klasycznych problemów automatyki i robotyki — sterowania odwróconym wahadłem — i łączy w sobie zagadnienia:

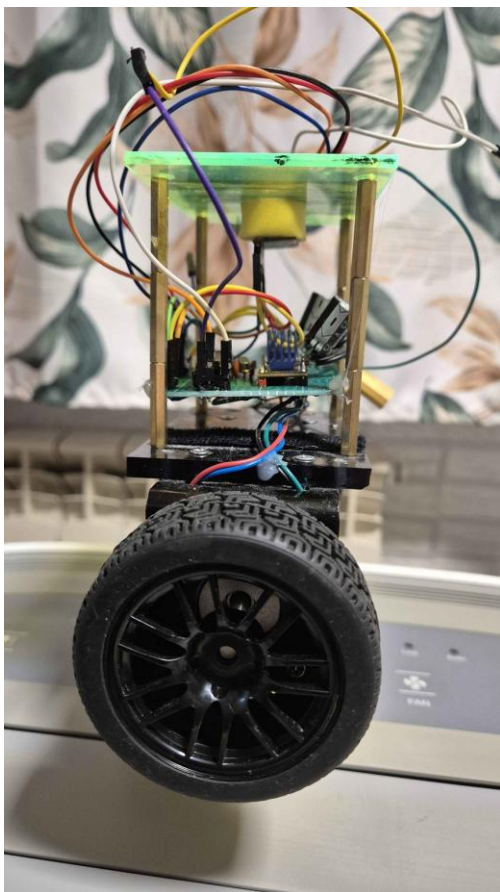
- mechaniki (rozkład mas, konstrukcja nośna, napęd),
- elektroniki (zasilanie, czujniki, sterowniki mocy, mikrokontroler),
- przetwarzania sygnałów (fuzja danych z akcelerometru i żyroskopu),
- teorii sterowania (synteza i strojenie regulatora PID),
- programowania systemów wbudowanych (praca w czasie rzeczywistym, przerwania sprzętowe, bezpośredni dostęp do rejestrów).

Zakres dokumentu obejmuje całość rozwiązania: od podstaw teoretycznych, przez dobór i opis komponentów, konstrukcję mechaniczną oraz elektronikę, aż po oprogramowanie sterujące, procedurę uruchomienia i wyniki testów.

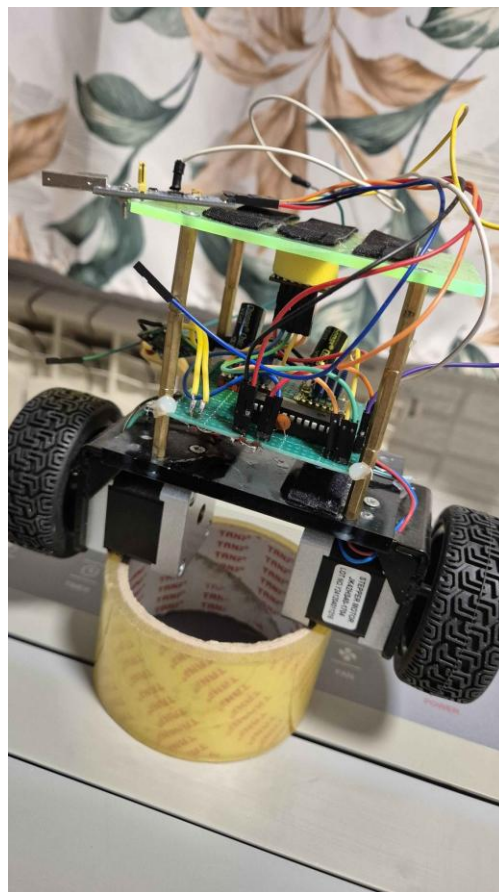
1.2. Idea robota samobalansującego

Robot stanowi fizyczną realizację odwróconego wahadła na wózku (wheeled inverted pendulum). W odróżnieniu od zwykłego wahadła, którego stan dolny jest stabilny, wahadło odwrócone ma środek masy powyżej punktu podparcia (osi kół) i jego pozycja pionowa jest punktem równowagi niestabilnej. Najmniejsze wychylenie powoduje narastający moment obracający korpus w stronę upadku.

Stabilizacja polega na tym, że układ regulacji podjeżdża kołami pod środek masy — gdy robot pochyła się do przodu, koła ruszają do przodu, aby ponownie ustawić oś kół pod środkiem ciężkości (i odwrotnie). Jest to ta sama zasada, dzięki której człowiek balansuje kijem ustawionym pionowo na dłoni.



a) Widok z boku — konstrukcja dwupokładowa (na podstawie pomocniczej)



b) Widok z przodu - symetryczny układ dwóch kół

Rysunek 1. Zbudowany dwukołowy robot samobalansujący

1.3. Założenia projektowe i wymagania

Wymaganie	Opis
Utrzymanie pionu	autonomiczne balansowanie na dwóch kołach bez podpór
Odporność na zakłócenia	powrót do równowagi po niewielkim trąceniu
Czujnik kąta	jednostka inercyjna 6-osiowa (akcelerometr + żyroskop)
Napęd	dwa niezależne silniki krokowe z mikrokrokiem
Sterowanie	cyfrowy regulator PID w pętli czasu rzeczywistego
Zasilanie	autonomiczne, z pakietu akumulatorów Li-ion
Bezpieczeństwo	wykrycie upadku i odcięcie napędu

Tabela 1. Założenia funkcjonalne i pozafunkcjonalne projektu

Podstawowe parametry zbudowanego robota zestawiono w poniższej tabeli.

Parametr	Wartość
Wysokość (bez kół)	ok. 14 cm
Wysokość (z kołami)	ok. 15 cm
Szerokość (z kołami)	ok. 20 cm
Rozstaw kół	ok. 15,5 cm
Średnica kół	ok. 7 cm
Masa (bez zasilania)	ok. 400 g
Masa (z baterią 9 V)	ok. 445 g
Położenie środka ciężkości	orientacyjnie w okolicy dolnej płyty (pleksy)

Tabela 2. Parametry gabarytowe zbudowanego robota

Masa robota bez źródła zasilania wynosi ok. 400 g; w wersji finalnej, zasilanej baterią 9 V (≈ 45 g), masa całkowita to ok. 445 g. Jakość regulacji potwierdzają zarejestrowane przebiegi (rozd. „Testy i wyniki”): robot utrzymuje pion z odchyleniem rzędu $\pm 0,6^\circ$ na gładkiej powierzchni.

2. Podstawy teoretyczne

2.1. Model odwróconego wahadła

Robota można przybliżyć modelem odwróconego wahadła o masie skupionej m w środku ciężkości oddalonym o l od osi kół. Po linearyzacji wokół pozycji pionowej ($\theta \approx 0$) równanie ruchu ma postać:

$$\ddot{\theta} = (g/l) \cdot \theta - (l/l) \cdot \ddot{x} \quad (2.1)$$

gdzie θ jest kątem pochylenia, g — przyspieszeniem ziemskim, a $\ddot{x}$ — przyspieszeniem poziomym punktu podparcia (kół). Dodatni współczynnik g/l przy θ świadczy o niestabilności układu otwartego: odchylenie samoistnie narasta wykładniczo. Zadaniem regulatora jest takie sterowanie przyspieszeniem kół $\ddot{x}$ (a w praktyce ich prędkością), aby sprowadzić θ do wartości zadanej.

2.2. Pomiar kąta — jednostka inercyjna i filtr komplementarny

Do estymacji kąta pochylenia wykorzystano czujnik MPU-6050 zawierający 3-osiowy akcelerometr i 3-osiowy żyroskop. Każdy z czujników mierzy kąt z innymi ograniczeniami:

- **Akcelerometr** mierzy kierunek wektora grawitacji, więc daje bezwzględny kąt pochylenia, lecz jest wrażliwy na drgania i przyspieszenia liniowe (duży szum przy ruchu).
- **Żyroskop** mierzy prędkość kątową; jej całkowanie daje gładki kąt o niskim szumie, ale obarczony dryfem narastającym w czasie.

Oba źródła łączy filtr komplementarny, który dla szybkich zmian ufa żyroskopowi, a w długim horyzoncie koryguje dryf akcelerometrem:

$$\theta(k) = \alpha \cdot (\theta(k-1) + \omega \cdot \Delta t) + (1 - \alpha) \cdot \theta_{acc}, \quad \alpha = 0,98 \quad (2.2)$$

Kąt z akcelerometru wyznaczany jest z dwóch osi:

$$\theta_{acc} = \text{atan2}(a_y, a_z) \cdot 180 / \pi \quad (2.3)$$

Stała czasowa filtru wynosi $\tau = \alpha/(1-\alpha) \cdot \Delta t = (0,98/0,02) \cdot 4 \text{ ms} \approx 0,20 \text{ s}$; poniżej tego czasu dominuje żyroskop, powyżej — akcelerometr.

2.3. Regulator PID

Sygnał sterujący wyznacza dyskretny regulator PID, którego prawo sterowania ma klasyczną postać:

$$u(t) = K_p \cdot e(t) + K_i \cdot \int e(\tau) d\tau + K_d \cdot de(t)/dt, \quad e = \theta_{zad} - \theta \quad (2.4)$$

W implementacji zastosowano dwie istotne modyfikacje praktyczne:

1. **Człon różniczkujący liczony z pomiaru** (derivative on measurement) — zamiast różniczkować uchyb, użyto wprost prędkości kątowej z żyroskopu ($d\theta/dt \approx \omega$). Eliminuje to gwałtowne „kopnięcia” członu D przy skokowej zmianie wartości zadanej.
2. **Ograniczenie nasycenia całki** (anti-windup) — całka jest przycinana do zakresu ± 25 , co zapobiega jej „rozbieganiu się” gdy robot leży lub jest przytrzymywany.

Parametr	Wartość	Opis
Kp	65,0	wzmocnienie proporcjonalne
Ki	0,4	wzmocnienie całkujące
Kd	1,0	wzmocnienie różniczkujące (na żyroskopie)
θ_{zad}	5,6°	kąt równowagi (TARGET_ANGLE) — korekta niesymetrii środka masy
α	0,98	współczynnik filtru komplementarnego
LPF (czł. D)	0,30	filtr dolnoprzepustowy członu różniczkującego
limit całki	± 25	anti-windup
limit upadku	35°	próg odcięcia napędu
f_p	250 Hz	częstotliwość pętli regulacji

Tabela 3. Nastawy regulatora i parametry pętli sterowania (z firmware)

2.4. Sterowanie silnikami krokowymi

Napęd realizują silniki krokowe, w których obrót następuje skokowo — o stały kąt 1,8° na pełny krok (200 kroków na obrót). Prędkość obrotowa jest wprost proporcjonalna do częstotliwości impulsów krokowych podawanych na wejście STEP sterownika. W projekcie prędkość zadaną u przeliczono na okres impulsu:

$$T_{\text{krok}} [\mu\text{s}] = 100000 / |u|, \quad T_{\text{krok}} \in (40; 20000) \mu\text{s} \quad (2.5)$$

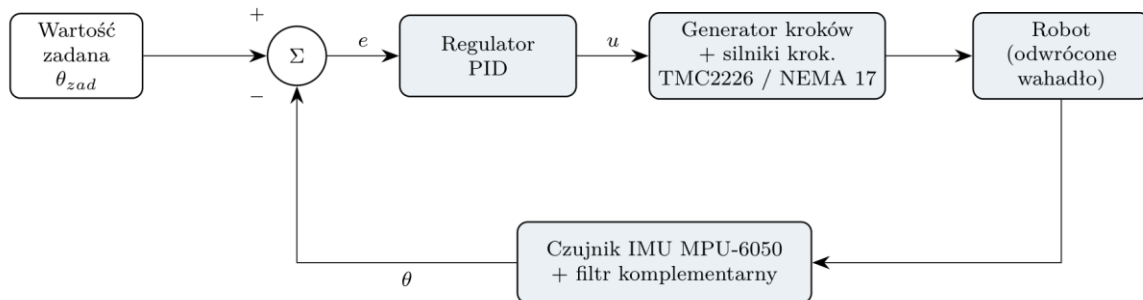
co odpowiada częstotliwości kroków od 50 Hz (powolny ruch) do 25 kHz (prędkość maksymalna). Ograniczenie dolne okresu wynika z maksymalnej prędkości, jaką silnik jest w stanie podążać bez utraty kroków; ograniczenie górne zapobiega zbyt rzadkim impulsom przy prędkości bliskiej zeru.

Sterowniki TMC2226 pracują w trybie STEP/DIR z mikrokrokiem, dzięki czemu ruch jest płynny i cichy (technologia StealthChop), co jest korzystne dla stabilności balansowania (mniej drgań przenoszonych na czujnik).

3. Architektura systemu

3.1. Schemat blokowy układu regulacji

Schemat blokowy (Rysunek 2) przedstawia strukturę pętli sprzężenia zwrotnego. Czujnik IMU mierzy stan korpusu, filtr komplementarny estymuje kąt, regulator PID wyznacza prędkość, a generator kroków zamienia ją na impulsy dla silników. Ruch kół zmienia pochylenie robota, zamykając pętlę.



Rysunek 2. Schemat blokowy zamkniętej pętli regulacji balansowania

3.2. Wykaz komponentów (BOM)

Poniższa tabela zawiera wykaz elementów użytych w układzie.

#	Element	Il.	Uwagi / parametry
1	Mikrokontroler ATmega328P-U (DIP)	1	DIP-28, rdzeń AVR 8-bit, 16 MHz (kwarce zewn.)
2	Czujnik IMU MPU-6050 (GY-521)	1	akcelerometr + żyroskop, magistrala I ² C, adres 0x68
3	Sterownik silnika TMC2226 (BIGTREETECH v1.0)	2	moduł StepStick, STEP/DIR, mikrokrok, radiator
4	Silnik krokowy NEMA 17 JK42HS40-1704	2	200 kr./obr. (1,8°), 2,8 V / 1,7 A na fazę, moment 0,4 N·m, dwubiegunowy
5	Wspornik silnika NEMA 17 (typ L)	2	stalowy, kątowny
6	Koła RC 1:10 (opona szosowa + felga)	2	guma, $\varnothing \approx 7$ cm, mocowanie hex 12 mm
7	Adapter piasty hex 12 mm	2	wał 5 mm → hex 12 mm, metalowy, śruby dociskowe
8	Bateria 9 V (6LR61)	1	źródło zasilania — wersja finalna, ≈ 45 g
9	Złącze zasilania 9 V (męskie)	1	przyłącze baterii 9 V — wersja finalna
10	Przetwornica step-down MP1584 (buck)	1	regulowana, 9 V → 5 V dla logiki
11	Konwerter USB→UART CP2102	1	programowanie + zasilanie 5 V podczas testów
12 †	Ogniwa 18650 Li-ion INR18650-F1HR	3	3350 mAh, 3S, $\approx 11,1$ V — wariant prototypowy
13 †	Złącze XT60	1	przyłącze pakietu 18650 — nie w wersji finalnej
14 †	Wyłącznik kołyskowy (DPDT)	1	odłączanie pakietu — nie w wersji finalnej
15	Kondensator elektrolit. 1000 μ F/25 V	2	filtrowanie zasilania silników (po jednym na sterownik)
16	Kondensator ceramiczny 100 nF (104)	≥ 2	odsprężanie zasilania
17	Kwarce 16 MHz + 2×22 pF	1	rezonator taktujący MCU
18	Płyta uniwersalna PCB 9×15 cm (jednostronna)	1	docięta na wymiar korpusu, montaż punkt-punkt
19	Przewody silikonowe 22 AWG	wg potrz.	elastyczne połączenia mocy/sygnali
20	Tuleje dystansowe mosiężne (hex) 25 mm	kpl.	słupki nośne między pokładami
21	Plekso górna — zielona wylewana, UV-odporna	1	3 mm, docięta z arkusza 200×200 mm
22	Plekso dolna — dymiona grafitowa	1	6 mm, docięta z arkusza

Tabela 4. Wykaz materiałowy (Bill of Materials)

Uwagi do wykazu: opona RC 1:10 ma szerokość ok. 26 mm; rezystory podciągające magistrali I²C są wbudowane w moduł GY-521 (MPU-6050), więc nie dodawano ich osobno; ustawienia sterowników TMC2226 (V_{ref}, mikrokrok, piny EN/MS1/MS2) zestawiono w tabeli konfiguracji sterowników; docelowe zasilanie opisano w sekcji „3.4. Zasilanie”.

3.3. Galeria kluczowych komponentów



a) ATmega328P-PU (DIP-28)



b) Moduł IMU MPU-6050



c) Silnik NEMA 17 JK42HS40-1704



d) Wspornik silnika (typ L)



e) Adaptery piast hex 12 mm



f) Koła RC z oponami

Rysunek 3. Główne komponenty mechaniczne i elektroniczne robota

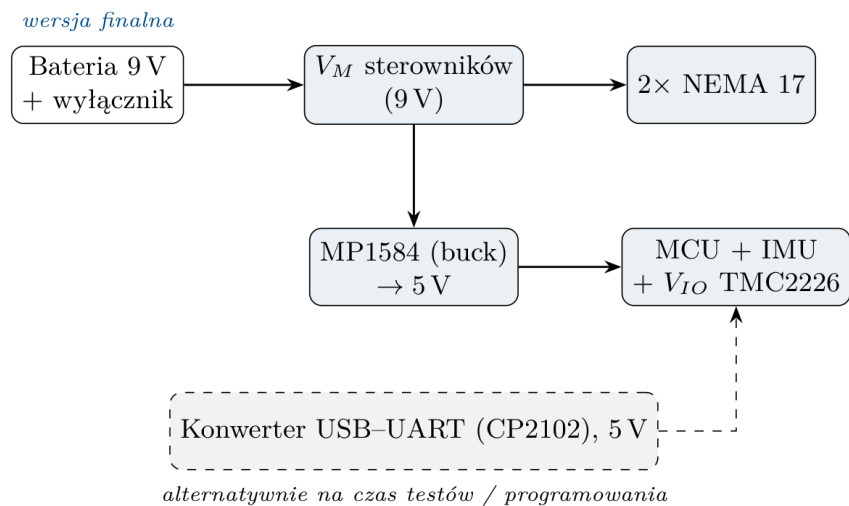
3.4. Zasilanie

Wersja finalna zasilana jest baterią 9 V dołączaną przez dedykowane złącze. Napięcie 9 V trafia bezpośrednio na wejścia mocy V_M sterowników TMC2226 (zasilanie silników), a równolegle przez przetwornicę obniżającą MP1584 jest sprowadzane do 5 V dla logiki (mikrokontroler, IMU oraz zasilanie cyfrowe V_{IO} sterowników). Przy każdym sterowniku zastosowano kondensator 1000 µF/25 V buforujący impulsowy pobór prądu silników.

Na czas testów i programowania robot zasilany jest z konwertera USB→UART CP2102 (5 V) — ten sam przewód służy do wgrywania firmware i podglądu telemetry, co zapewnia „czyste” zasilanie podczas strojenia.

Wcześniej testowano pakiet 3S 18650 (3 ogniwa, ≈ 11,1 V, złącze XT60, wyłącznik) — wariant prototypowy, przy którym robot nie pracował stabilnie (zapady napięcia pod obciążeniem);

zrezygnowano z niego na rzecz baterii 9 V. Dystrybucję zasilania wersji finalnej przedstawia poniższy schemat.



Rysunek 4. Dystrybucja zasilania w wersji finalnej (bateria 9 V); przerywaną linią zaznaczono zasilanie z CP2102 używane podczas testów

3.5. Połączenia elektryczne i mapa wyprowadzeń

Mapa wyprowadzeń (Tabela 5) zestawia przyporządkowanie wyprowadzeń mikrokontrolera (numer fizyczny obudowy DIP, notacja Arduino oraz port AVR) do sygnałów sterujących. Mikrokontroler programowany jest oraz komunikuje się przez konwerter USB→UART (linie RXD/TXD), a pin RESET podciągnięto rezystorem do 5 V. Strukturę połączeń całego układu ilustruje Rysunek 5.

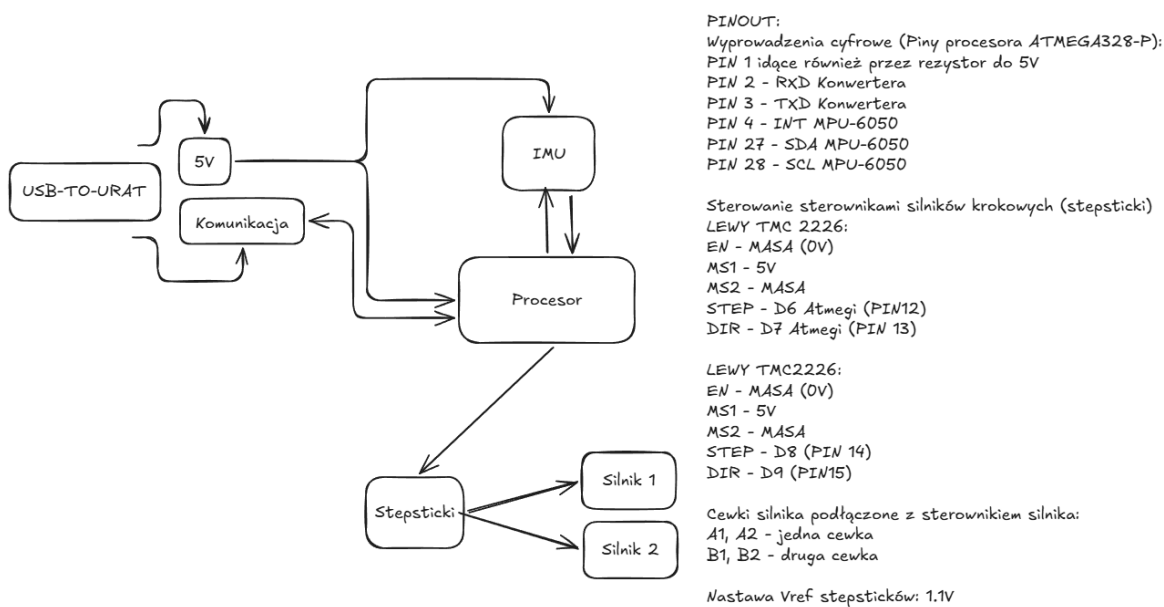
Pin fiz.	Arduino	Port	Kierunek	Sygnał	Połączenie
1	RESET	PC6	wejście	RST (akt. niski)	rezystor podciągający do 5 V
2	D0	PD0	wejście	RXD	RXD konwertera USB→UART
3	D1	PD1	wyjście	TXD	TXD konwertera USB→UART (debug 115200 Bd)
4	D2	PD2	wejście	INT	przerwanie z MPU-6050 (INT)
12	D6	PD6	wyjście	STEP_L	STEP lewego TMC2226
13	D7	PD7	wyjście	DIR_L	DIR lewego TMC2226
14	D8	PB0	wyjście	STEP_R	STEP prawego TMC2226
15	D9	PB1	wyjście	DIR_R	DIR prawego TMC2226 (kierunek odwrócony)
27	A4	PC4	I/O	SDA	dana I ² C → MPU-6050
28	A5	PC5	I/O	SCL	zegar I ² C → MPU-6050

Tabela 5. Mapa wyprowadzeń ATmega328P

Konfigurację obu modułów TMC2226 (w trybie STEP/DIR, bez UART) zestawiono w Tabeli 6. Sterowniki pracują przy mikrokroku 1/32 ($MS1 = 5\text{ V}$, $MS2 = \text{GND} \Rightarrow 6400$ mikrokroków na obrót) i ustawionym napięciu odniesienia $V_{\text{ref}} = 1,1\text{ V}$. Wejścia EN zwarto do masy, co trwale uaktywnia stopnie mocy. Cewki silnika podłączono parami: A1/A2 — pierwsza cewka, B1/B2 — druga.

Sygnal	Lewy sterownik	Prawy sterownik
EN	GND (stopień mocy aktywny)	GND (stopień mocy aktywny)
MS1	5 V	5 V
MS2	GND	GND
Mikrokrok	1/32 (6400 kr./obróć)	1/32 (6400 kr./obróć)
STEP	D6 (PD6, pin 12)	D8 (PB0, pin 14)
DIR	D7 (PD7, pin 13)	D9 (PB1, pin 15)
Vref	1,1 V	1,1 V
Cewki	A1/A2 — cewka 1	B1/B2 — cewka 2

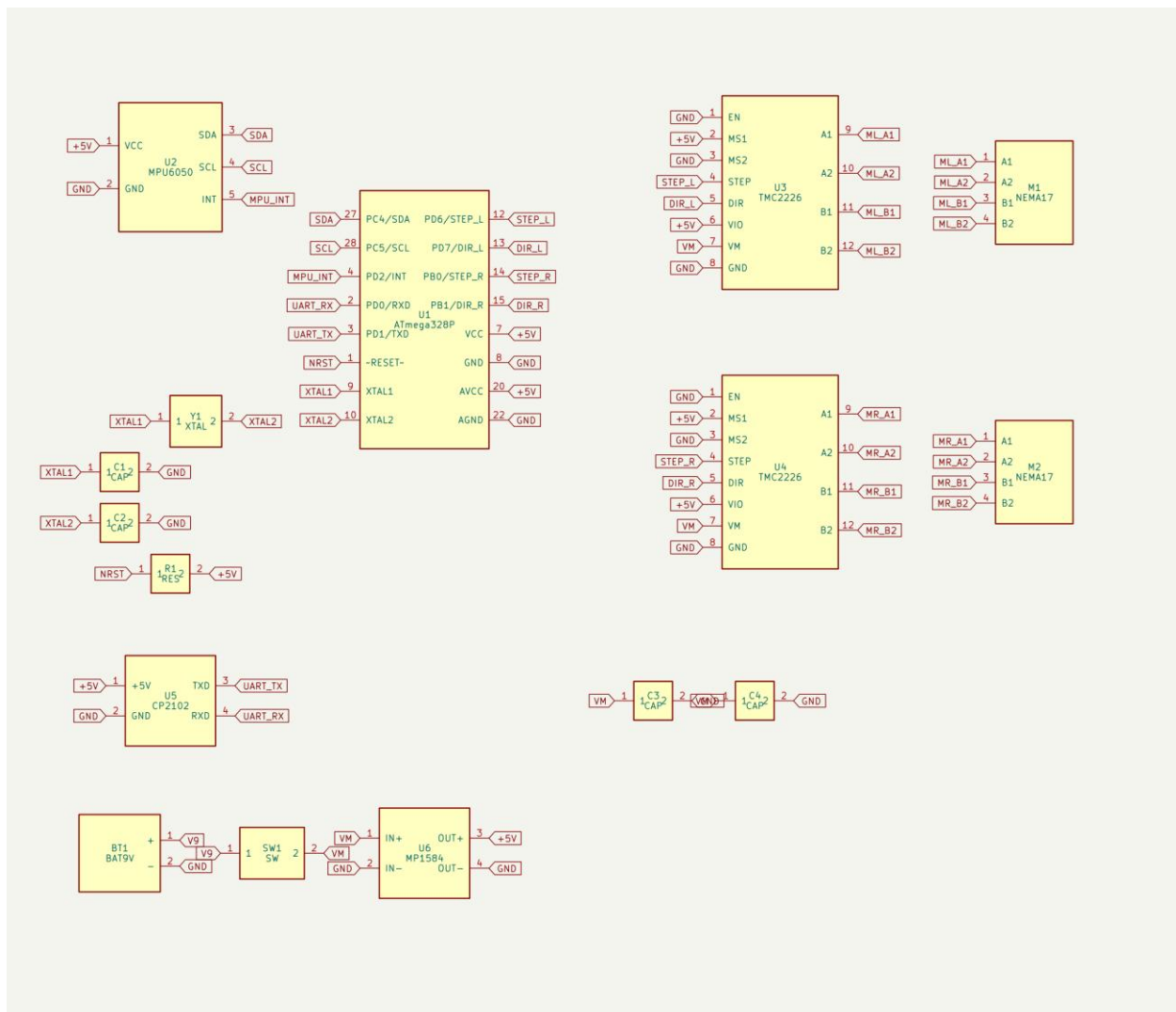
Tabela 6. Konfiguracja sterowników silników krokowych TMC2226



Rysunek 5. Schemat połączeń i wykaz wyprowadzeń (materiał projektowy autora)

3.6. Schemat ideowy

Na podstawie szkicu projektowego i firmware wykonano schemat ideowy całego układu w programie KiCad (rysunek poniżej). Obejmuje on mikrokontroler, czujnik IMU, dwa sterowniki TMC2226 z silnikami, rezonator kwarcowy oraz drzewo zasilania. Połączenia zrealizowano globalnymi etykietami sieci (końcówki o tej samej nazwie sieci, np. +5V, GND, SDA, STEP_L, VM, są ze sobą połączone). Edytowalny projekt udostępniono w katalogu kicad/ (pliki robot.kicad_pro, robot.kicad_sch).



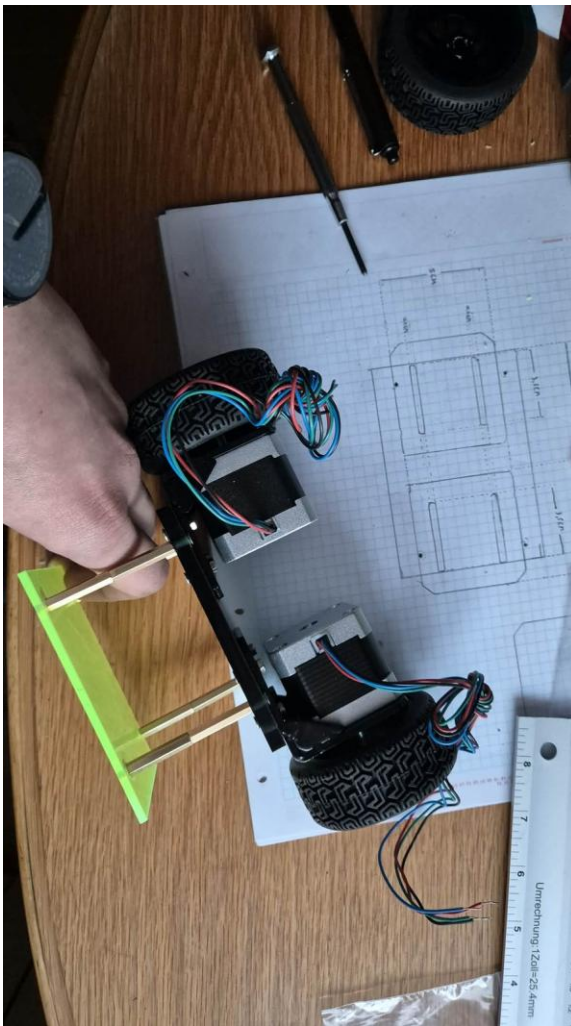
Rysunek 6. Schemat ideowy robota wykonany w KiCad (połączenia wg nazw sieci)

Uwaga: symbole na schemacie są celowo uproszczone (prostokątne, z połączeniami przez nazwy sieci); przy przenoszeniu układu na dedykowane PCB warto użyć symboli bibliotecznych oraz dodać komplet kondensatorów odsprężających przy każdym układzie scalonym.

4. Konstrukcja mechaniczna

Konstrukcję nośną robota stanowi układ dwupokładowy: dwie płyty z pleksy połączone mosiężnymi tulejami dystansowymi (hex) o długości 25 mm. Płytę dolną wykonano z pleksy dymionej grafitowej o grubości 6 mm (sztywna baza, do której przykręcono silniki), a płytę górną z pleksy zielonej wylewanej, odpornej na UV, o grubości 3 mm; obie docięto z arkusza. Rama jest sztywna, łatwa w modyfikacji i pozwala розміścić masę (elektronika, IMU) na różnych poziomach, co wpływa na położenie środka ciężkości — kluczowy parametr przy balansowaniu.

Silniki krokowe NEMA 17 zamocowano nisko, na wspornikach typu L, symetrycznie po obu stronach. Koła RC (1:10, $\varnothing \approx 7$ cm) osadzono na wałach silników za pomocą metalowych adapterów piast hex 12 mm ze śrubami dociskowymi. Gabaryty: wysokość ok. 14 cm (bez kół) / 15 cm (z kołami), szerokość ok. 20 cm, rozstaw kół ok. 15,5 cm, masa ok. 400 g (bez akumulatorów). Środek ciężkości znajduje się orientacyjnie w okolicy dolnej płyty. Im wyżej środek masy, tym wolniejsze przewracanie i łatwiejsza regulacja.

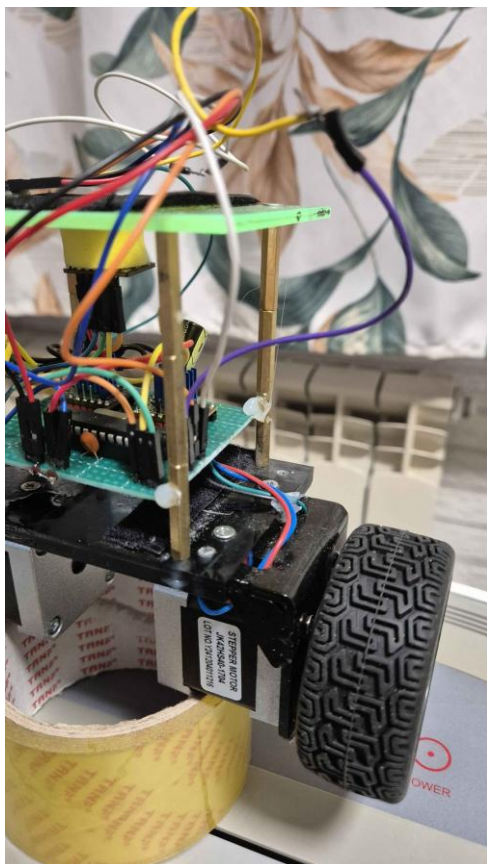


a) Para silników NEMA 17 z kołami i odręczny szkic ramy na papierze milimetrowym

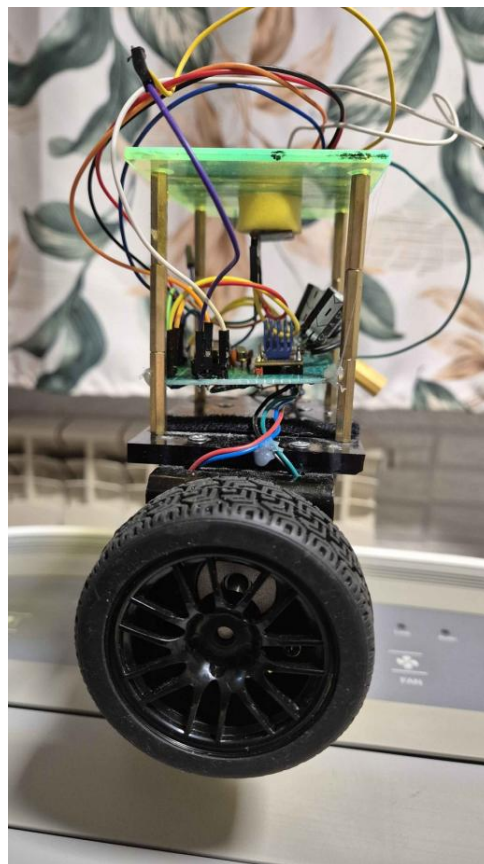


b) Komponenty na stanowisku montażowym

Rysunek 7. Etap projektowania i kompletowania konstrukcji mechanicznej



*a) Widok z boku — dwa pokłady pleksy, tuleje mosiężne,
silnik JK42HS40-1704*

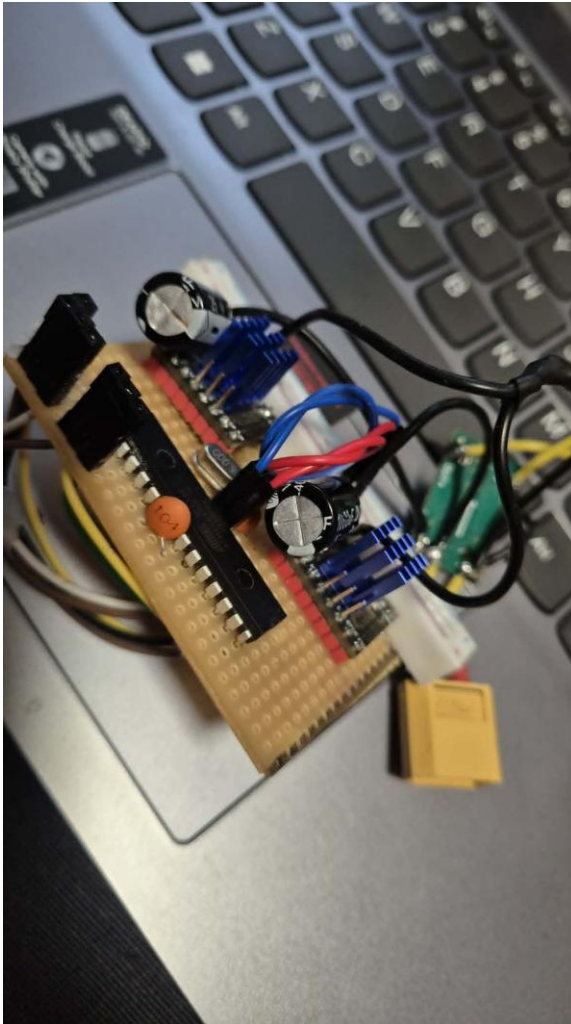


b) Widok od czola — symetryczny rozstaw kół RC

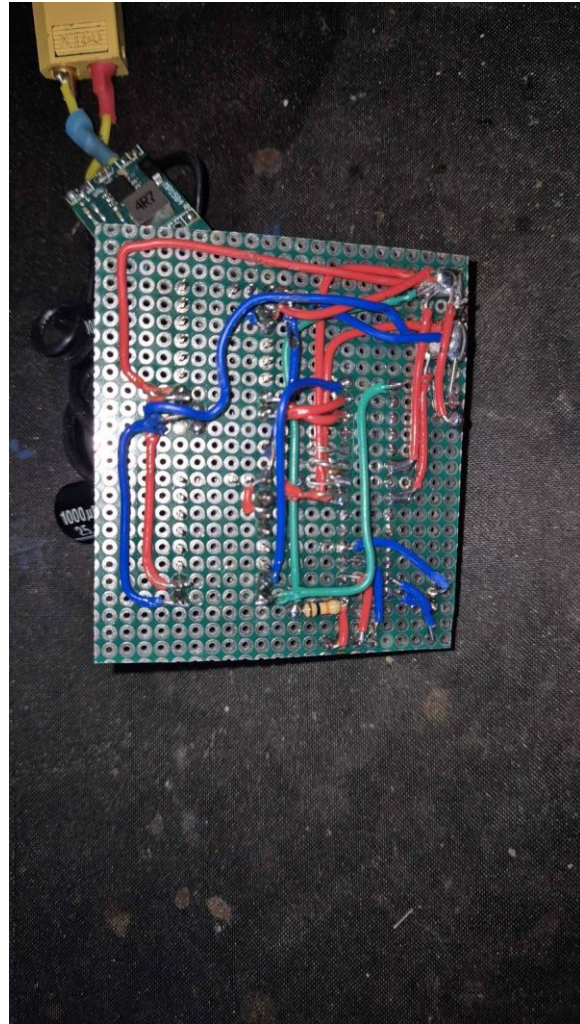
Rysunek 8. Zbudowana konstrukcja mechaniczna robota

5. Elektronika

Elektronikę zmontowano metodą przewlekaną na płytce uniwersalnej (perfboard); połączenia wykonano punkt-punkt lutowanymi przewodami. Centralnym elementem jest mikrokontroler ATmega328P-PU w podstawce, taktowany zewnętrznym kwarem 16 MHz. Po obu stronach znajdują się gniazda listew, w które wpięto moduły sterowników TMC2226 z radiatorami; przy każdym sterowniku umieszczono kondensator 1000 μ F buforujący zasilanie silnika.

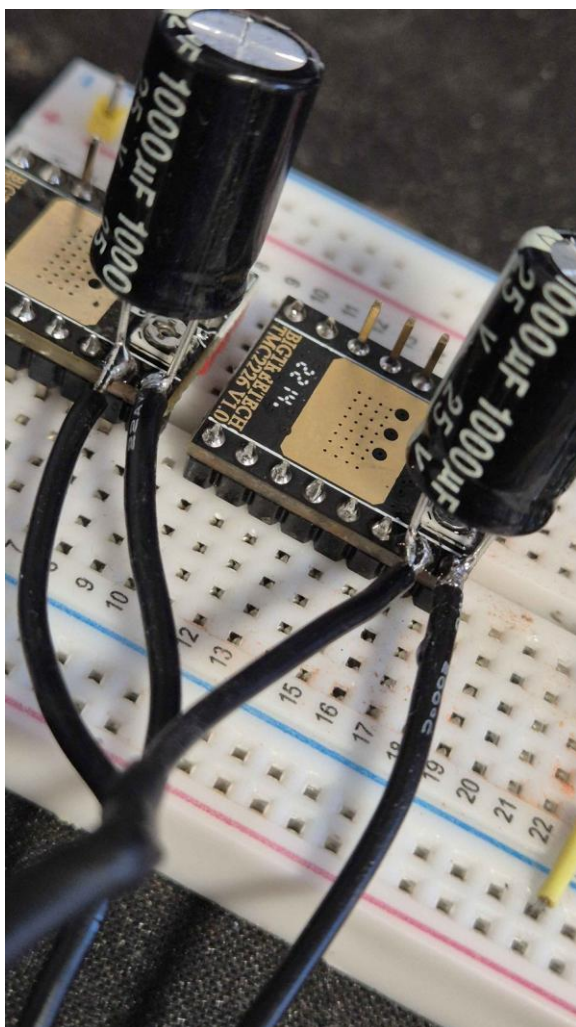


a) Płyta główna — widok z góry: MCU, kwarc, dwa TMC2226, kondensatory 1000 μ F, XT60

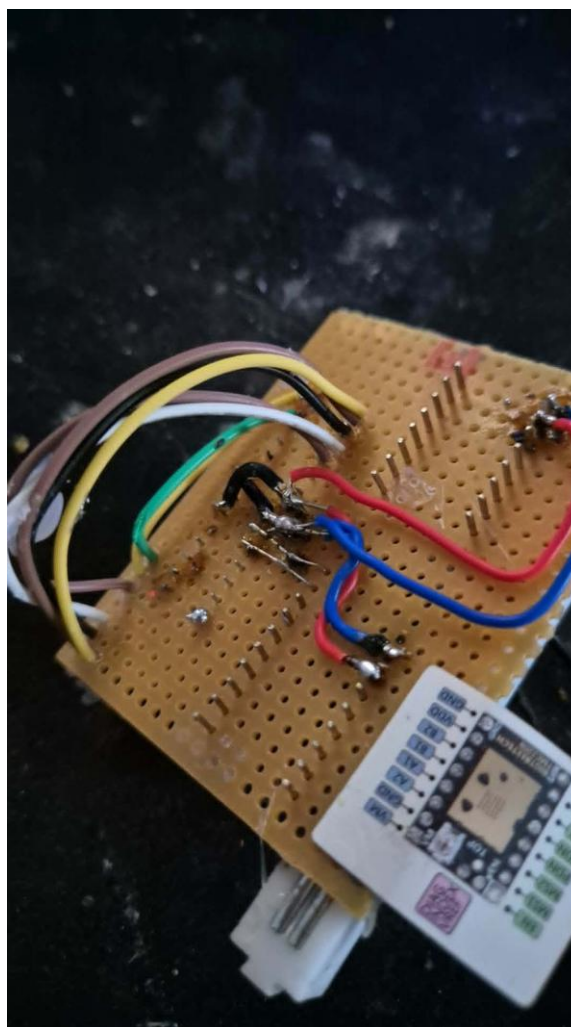


b) Strona lutowania — połączenia punkt-punkt, moduł IMU i przylącze XT60

Rysunek 9. Płyta sterująca robota wykonana na płytce uniwersalnej

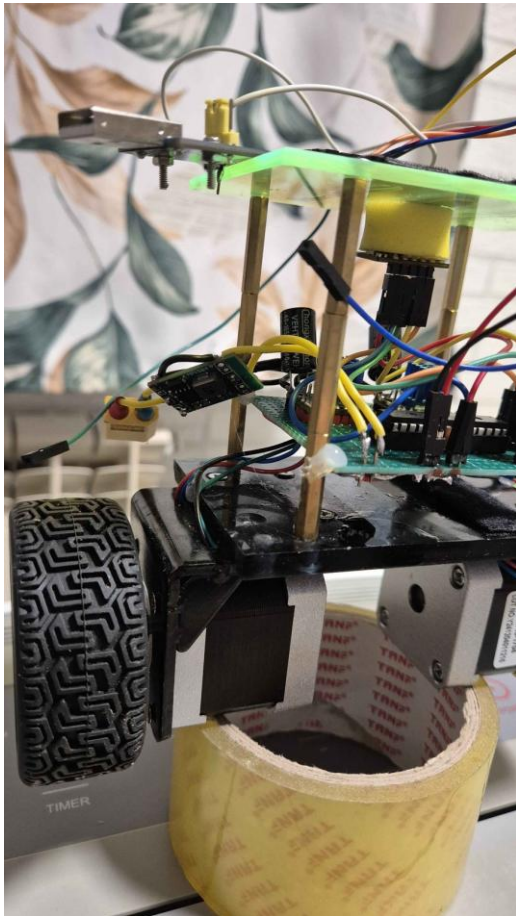


a) Sterowniki TMC2226 na płytce stykowej (testy) z kondensatorami 1000 μ F/25 V

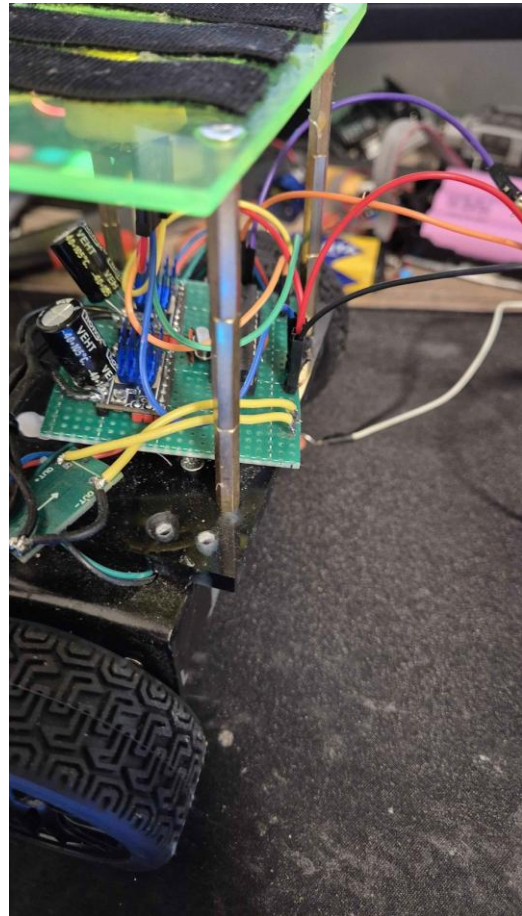


b) Etykieta wyprowadzeń modułu TMC2226 (GND, VDD, B2, B1, A1, A2, V_M + sterowanie)

Rysunek 10. Sterowniki silników krokowych TMC2226 i ich wyprowadzenia



a) Zmontowana elektronika: przetwornica MP1584, kondensatory, ATmega328P, TMC2226; u góry IMU



b) Detal: przetwornica step-down (OUT+/OUT-) i kondensatory filtrujące

Rysunek 11. Elektronika w docelowym montażu na konstrukcji robota

5.1. Sterownik TMC2226 — uwagi

Moduły TMC2226 to ciche sterowniki silników krokowych z mikrokrokiem i regulacją prądu. Istotne dla poprawnej pracy:

- **Prąd fazowy** ustawiany potencjometrem V_{ref} — w projekcie ustawiono $V_{ref} = 1,1 \text{ V}$ (z zapasem względem znamionowych $1,7 \text{ A}$ silnika, co ogranicza nagrzewanie).
- **Mikrokrok** (piny MS1/MS2) — wpływa na płynność i maksymalną prędkość; tu MS1 = 5 V, MS2 = GND, czyli 1/32 (6400 mikrokroków na obrót), co zapewnia bardzo płynny, cichy ruch.
- **Wymóg czasowy STEP** — impuls musi trwać dłużej niż 100 ns; w firmware zapewniono to przez wstawienie kilku instrukcji nop między ustawieniem a skasowaniem linii STEP.
- **Odsprężanie zasilania** — kondensator $1000 \mu\text{F}$ przy V_M chroni przed przepięciami od indukcyjności silnika.

6. Oprogramowanie sterujące

Oprogramowanie napisano w środowisku Arduino (rdzeń AVR), jednak z licznymi optymalizacjami niskopoziomowymi: bezpośrednim dostępem do portów (PORTB, PORTD) oraz sprzętowym generowaniem impulsów krokowych w przerwaniu licznika Timer1. Dzięki temu krytyczny czasowo proces krokowania jest niezależny od czasu wykonania głównej pętli.

6.1. Struktura programu

Program składa się z trzech zasadniczych części:

3. `setup()` — inicjalizacja I²C, portów, licznika Timer1 (tryb CTC), konfiguracja MPU-6050 i kalibracja żyroskopu;
4. `loop()` — pętla regulacji 250 Hz: odczyt IMU, filtr komplementarny, regulator PID, ograniczenie prędkości, wysłanie komendy do silników, zabezpieczenia;
5. `ISR(TIMER1_COMPA_vect)` — przerwanie generujące impuls STEP na obu silnikach jednocześnie.

6.2. Pętla regulacji 250 Hz

Każda iteracja jest „bramkowana” czasowo: pętla czeka, aż upłynie założony slot 4 ms, a faktyczny krok czasowy Δt mierzony jest funkcją `micros()` i używany w całkowaniu/różniczkowaniu — co czyni regulację odporną na drobne wahania czasu wykonania.

Listing: Filtr komplementarny i wyznaczenie uchybu (fragment `loop()`)

```
// --- Filtr komplementarny ---
float accAngle = atan2((float)accY, (float)accZ) * 180.0f / PI;
compAngle = COMP_ALPHA * (compAngle + gyroX * dt) + (1.0f - COMP_ALPHA) * accAngle;

// --- Setpoint i blad ---
float setpoint = TARGET_ANGLE + angleTrim;
float error = setpoint - compAngle;
```

Listing: Regulator PID z anti-windup i członem D na pomiarze

```
// --- PID ---
errorIntegral += error * dt;
errorIntegral = constrain(errorIntegral, -I_LIMIT, I_LIMIT); // anti-windup

float dRaw = -gyroX; // czlon D z predkosci katowej
dTermFilt += D_LPF_ALPHA * (dRaw - dTermFilt); // filtr dolnoprzepustowy

float output = Kp * error + Ki * errorIntegral + Kd * dTermFilt;
```

6.3. Generacja kroków i konwersja prędkości

Funkcja `setMotorSpeed()` ustala kierunek (linie DIR), a następnie przelicza zadaną prędkość na okres licznika. Ponieważ Timer1 pracuje z preskalerem 8 (takt 0,5 μ s), wartość rejestru

Listing: *Konwersja prędkość $\rightarrow$ okres kroku oraz aktualizacja Timer1*

Listing: *Przerwanie sprzętowe generujące impuls STEP na obu silnikach*

Listing: Wykrywanie upadku i odcięcie napędu

22

6.6. Rozszerzenie: automatyczny obrót co ~13 s

Opracowano wariant oprogramowania (plik balanser_obrot.ino, pełny listing w Dodatku B), w którym robot — nie przerywając balansowania — co ok. 13 s wykonuje obrót w lewo o 45° (mniejszy obrót mniej zaburza balans; parametry TURN_ANGLE_DEG, TURN_INTERVAL_MS). Wymagało to zmiany sposobu generowania kroków, tak aby każde koło mogło mieć inną prędkość (skręt = różnica prędkości kół).

Niezależne krokowanie kół (DDS). Zamiast jednego okresu wspólnego dla obu silników wprowadzono jedno przerwanie Timer1 o stałej częstotliwości $f_{ISR} = 25 \text{ kHz}$ oraz dwa akumulatory fazy (technika DDS / Bresenhama). W każdym przerwaniu do akumulatora danego koła dodawana jest jego zadana częstotliwość kroków; po przekroczeniu progu f_{ISR} generowany jest impuls STEP. Dzięki temu z jednego licznika uzyskuje się dwie niezależne częstotliwości krokowania. Aby przerwanie było maksymalnie krótkie (i nie zaburzało pętli regulacji ani magistrali I²C), użyto arytmetyki 16-bitowej, a kierunek obrotu (linie DIR) ustawiany jest poza przerwaniem. Zakres prędkości i nastawy regulatora pozostawiono takie same jak w wersji podstawowej (m.in. MAX_SPEED = 2500).

Listing: Generacja kroków obu kół z dwoma akumulatorami fazy (lekki ISR)

```
ISR(TIMER1_COMPA_vect) { // wywoływane STEP_ISR_HZ = 25000 razy/s
    if (!motorsEnabled) return;
    accl += rateMagL; // uint16, |predkosc| lewego koła
    if (accl >= STEP_ISR_HZ) { accl -= STEP_ISR_HZ; /* impuls STEP lewy */ }
    accR += rateMagR; // uint16, |predkosc| prawego koła
    if (accR >= STEP_ISR_HZ) { accR -= STEP_ISR_HZ; /* impuls STEP prawy */ }
}
```

Uwaga (historia poprawek): pierwsza wersja używała $f_{ISR} = 31250 \text{ Hz}$ i arytmetyki 32-bitowej — zbyt obciążało to mikrokontroler, przez co pętla regulacji nie nadążała i robot nie balansował. Po przejściu na lekkie, 16-bitowe przerwanie robot balansuje; aby ograniczyć drgania i upadek na końcu obrotu, przywrócono oryginalny zakres MAX_SPEED oraz wprowadzono rampę obrotu (płynne narastanie i wygaszanie prędkości różnicowej zamiast skokowej zmiany).

Składanie prędkości i harmonogram obrotu. Prędkość każdego koła jest sumą składowej balansu (wspólnej) i składowej obrotu (różnicowej): $v_L = v_{bal} - v_{turn}$, $v_R = v_{bal} + v_{turn}$. Liczbę mikrokroków potrzebną do obrotu o 45° wyznaczono z geometrii (rozstaw kół 15,5 cm, średnica koła 7 cm, 6400 mikrokroków/obrot):

$$N_{obr} = (\varphi / 180^\circ) \cdot ((b/2) / D) \cdot N_{mikro} \approx 1771 \text{ mikrokroków na koło} \quad (6.1)$$

Aby uniknąć szarpnięcia (i upadku) przy gwałtownym włączeniu/wyłączeniu obrotu, prędkość różnicowa turnCmd jest rampowana — płynnie narasta do TURN_RATE i płynnie wygasa się do zera:

Listing: Harmonogram obrotu z rampą i złożenie prędkości kół

```
if (!turning && (millis() - lastTurnMs) >= TURN_INTERVAL_MS) {
    turning = true; turnStepsLeft = TURN_STEPS; // start obrotu
}
```

```

float turnTarget = 0.0f;
if (turning) {
    // PRIORYTET BALANSU: krec sie tylko gdy robot stoi w miare pionowo
    if (fabs(error) < TURN_PITCH_PAUSE) turnTarget = TURN_DIR * TURN_RATE;
    turnStepsLeft -= fabs(turnCmd) * dt;          // licz wykonany luk wg AKTUALNEJ predkosci
    if (turnStepsLeft <= 0) { turning = false; lastTurnMs = millis(); }
}
float dTurn = TURN_RAMP * dt;                    // RAMPA: plynne narastanie/wygaszanie
turnCmd += constrain(turnTarget - turnCmd, -dTurn, dTurn);

float rateBal = cmdSpeed * SPEED_TO_RATE;
setMotorRates(rateBal - turnCmd, rateBal + turnCmd); // lewe wolniej, prawe szybciej

```

Przygotowanie do obrotu (wejście ze stabilnego stanu). Obrót rozpoczynany w chwili, gdy robot akurat się przechyla lub jedzie, nakłada zaburzenie na już zaburzony stan. Dlatego wprowadzono fazę uzbrojenia: po upływie `TURN_INTERVAL_MS` obrót staje się „uzbrojony”, lecz startuje dopiero, gdy robot jest pionowo i spokojny — $|e| < \text{TURN_READY_ANGLE}$, mała prędkość kątowa i mały sygnał prędkości — utrzymane przez `TURN_READY_TIME`. Gdy robot nie wyciszy się w zadanym czasie (`TURN_ARM_TIMEOUT_MS`), obrót wykonywany jest mimo to. Stany sygnalizuje telemetria: `st=0` (balans), `st=1` (przygotowanie), `st=2` (obróć).

Niezawodność przy cięższej górze. Zamontowanie pakietu/baterii na górnej płycie podnosi środek masy i moment bezwładności względem osi pionowej, przez co szybki obrót łatwiej wytrąca robota z pionu. Aby obrót działał pewnie, balansowi nadano proporcjonalny priorytet nad obrotem. Prędkość różnicowa obrotu jest skalowana zależnie od przechylenia: poniżej `TURN_PITCH_FULL` ($1,5^\circ$) — pełna, w przedziale do `TURN_PITCH_STOP` (4°) — liniowo maleje do zera, a powyżej `TURN_PITCH_EMERG` (7°) — natychmiast odcinana (awaryjnie). Dodatkowo zmniejszono prędkość obrotu (`TURN_RATE = 1200`), co ogranicza moment potrzebny do obrotu.

Utrzymanie kursu i sterowanie przód-tył. Różnicowanie prędkości kół pozwala dodatkowo sterować kursem (yaw) — opcjonalna pętla PD na żyroskopie osi Z (`ENABLE_YAW_HOLD`, wzmocnienia `KYAW_P`, `KYAW_D`) utrzymuje robota prosto i przeciwdziała samoczynnemu znoszeniu. Jazda przód-tył przy balansie polega nie na wprost zadawaniu prędkości kół, lecz na zadaniu niewielkiego pochylenia, za którym robot „goni”. Całka sygnału prędkości daje darmową odometrię, na której oparto opcjonalną pętlę utrzymania pozycji (`ENABLE_POS_HOLD`, anti-drift). Obie konfiguracje kompilują się dla Arduino Uno; ostateczne strojenie należy wykonać na sprzęcie.

7. Uruchomienie i strojenie

7.1. Procedura uruchomienia

Robot jest gotowy do użycia — firmware jest już wgrany, a nastawy regulatora zapisane, więc nie trzeba niczego konfigurować ani programować. Wystarczy:

6. Podłączyć zasilanie (w wersji bieżącej — przewód konwertera USB-UART CP2102, dostarczający 5 V).
7. Postawić robota nieruchomo w pionie na czas automatycznej kalibracji żyroskopu (ok. 2 s po starcie) — to jedyny warunek poprawnego startu. Po kalibracji robot zaczyna balansować.

Opcjonalnie można otworzyć monitor portu szeregowego (115200 Bd), aby obserwować telemetrię (m.in. `ang=`, `err=`).

Kalibracja punktu równowagi (TARGET_ANGLE). Punkt równowagi dwukołowego robota zależy od rozkładu masy — po dołożeniu baterii na górze sztywna wartość TARGET_ANGLE przestaje pasować, przez co robot ma stałe przechylenie i „odjeżdża”. Pewna metoda: przytrzymać robota dokładnie w równowadze (tam, gdzie nie chce się przewracać w żadną stronę), odczytać pole `ang=` z monitora i wpisać tę wartość jako TARGET_ANGLE w kodzie. W firmware dostępne są też opcje (domyślnie wyłączone) automatyzujące ten krok: AUTO_TARGET_AT_START (przechwycenie kąta trzymanego przy starcie) oraz AUTO_TRIM (wolne dostrajanie w trakcie pracy) — działają poprawnie tylko przy precyzyjnym dobraniu znaku, dlatego zaleca się najpierw metodę ręczną.

7.2. Strojenie regulatora PID

Zastosowane nastawy ($K_p = 65$, $K_i = 0,4$, $K_d = 1,0$; Tabela 3) dobrano eksperymentalnie. Gdyby zaszła potrzeba ponownego strojenia, zalecana kolejność (metoda inżynierska) jest następująca:

8. Postawić robota nieruchomo na czas kalibracji żyroskopu (ok. 2 s).
9. Wyzerować K_i i K_d . Zwiększać K_p , aż robot zacznie reagować i „walczyć” o pion (dopuszczalne lekkie oscylacje).
10. Dodać K_d w celu wytłumienia oscylacji (człon D działa jak tarcie).
11. Dodać niewielkie K_i , aby zlikwidować powolny dryf/odchyłkę statyczną.
12. Skorygować TARGET_ANGLE tak, aby odpowiadał rzeczywistemu położeniu równowagi (kątem, przy którym robot stoi bez tendencji do jazdy).

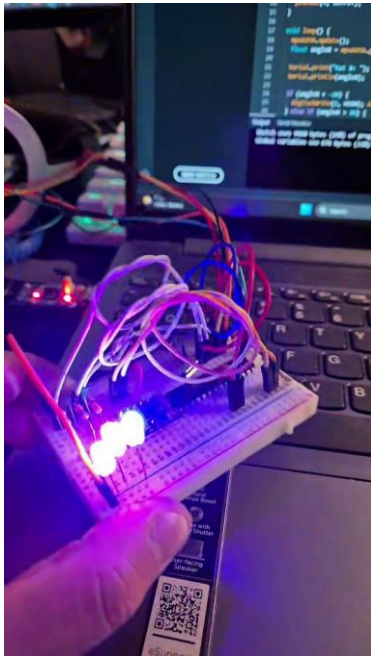
8. Testy i wyniki

8.1. Test odczytu IMU z wizualizacją na diodach LED

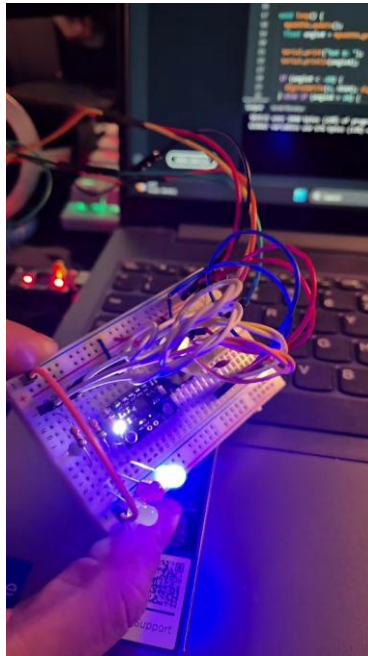
Na wczesnym etapie, przed integracją z silnikami, poprawność toru pomiarowego (MPU-6050 + filtr komplementarny) zweryfikowano niezależnie, sygnalizując odczyt pochylenia na diodach LED na płytce stykowej. Pochylanie układu powodowało proporcjonalną reakcję diod, a na monitorze szeregowym widoczne były wartości kąta i uchybu. Test ten potwierdził, że:

- komunikacja I²C z MPU-6050 działa poprawnie,
- filtr komplementarny zwraca stabilny, niezaszumiony kąt,
- znak i zakres pomiaru są zgodne z oczekiwaniem.

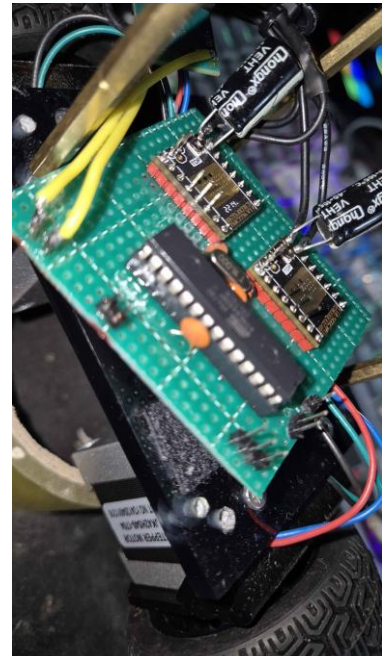
Uwaga: prezentowane nagranie obrazuje wyłącznie działanie czujnika na diodach — nie przedstawia jeszcze jazdy robota na kołach/silnikach.



a) Reakcja diody na pochylenie



b) Podgląd wartości na monitorze szeregowym



c) Płyta zamontowana między kołami

Rysunek 12. Walidacja toru pomiarowego IMU (kadry z nagrania) oraz docelowy montaż

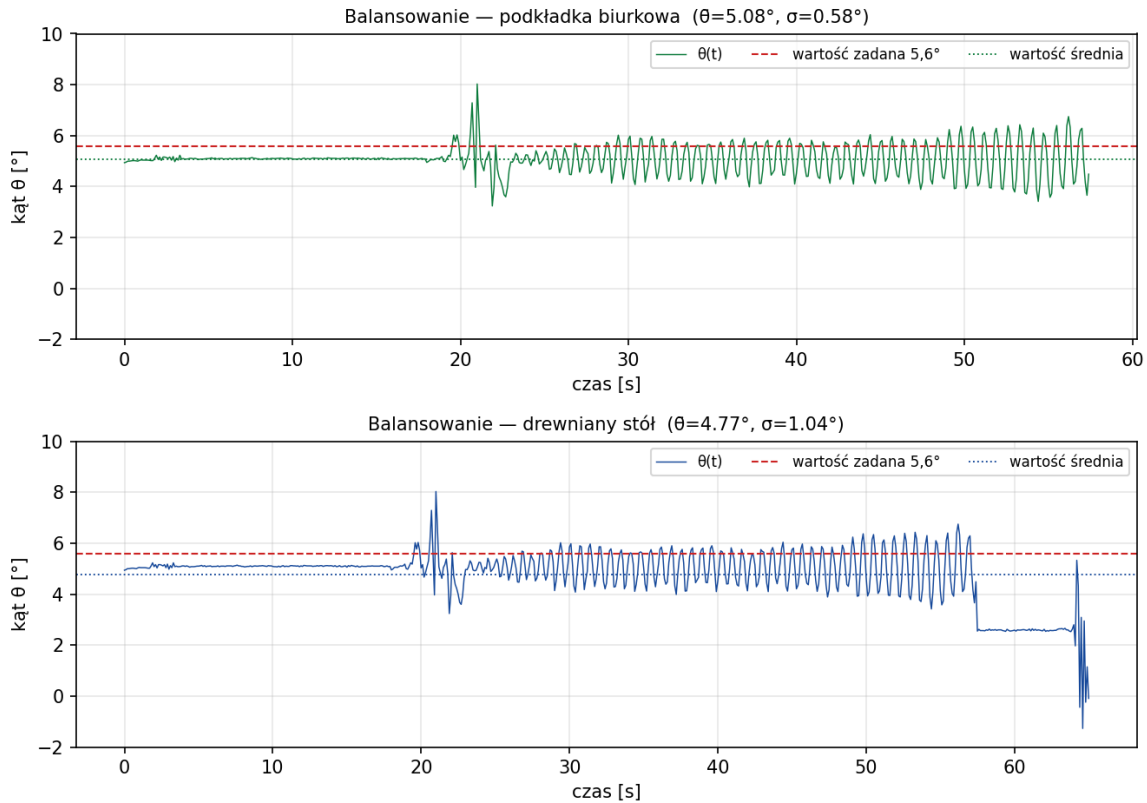
8.2. Telemetria — monitor szeregowy

Firmware co 25 iteracji (ok. 10 razy na sekundę) wypisuje stan regulacji, co ułatwia diagnostykę i strojenie:

```
ang=<kat> err=<uchyb> out=<wyjscie PID> spd=<predkosc> trim=<korekta>
```

8.3. Wyniki balansowania na różnych nawierzchniach

Zarejestrowano przebiegi telemetrii podczas balansowania robota na dwóch nawierzchniach: gładkiej podkładce biurkowej oraz drewnianym stole. Poniższy rysunek przedstawia estymowany kąt pochylenia θ w funkcji czasu (próbkowanie 10 Hz).



Rysunek 13. Estymowany kąt pochylenia podczas balansowania — porównanie dwóch nawierzchni. Linia przerywana: wartość zadana $5,6^\circ$; linia kropkowana: wartość średnia.

Obserwacje:

- **Robot utrzymuje równowagę** wokół punktu pracy: wartość średnia kąta wynosi $5,08^\circ$ (podkładka) oraz $4,77^\circ$ (stół), przy zadanej $5,6^\circ$.
- **Niewielka odchyłka statyczna** (poniżej wartości zadanej) — efekt małego wzmocnienia całkującego K_i .
- Po początkowej fazie spokojnej obserwuje się cykl graniczny (drgania ustalone) typowy dla balansowania na sztywnych kołach.
- Na drewnianym stole drgania są wyraźnie większe (odchylenie standardowe $1,04^\circ$ wobec $0,58^\circ$ na podkładce) — gładza, twardsza powierzchnia sprzyja poślizgowi kół i utrudnia regulację; pod koniec testu zarejestrowano duże wychylenie kończące się utratą równowagi.

Uwaga: przebieg dla drewnianego stołu obejmuje pojedynczy, ciągły test (usunięto błędnie dołączonego drugiego pomiaru z pierwotnego logu).

9. Napotkane problemy i wnioski

Wnioski techniczne

- **Zastosowanie silników krokowych** (zamiast silników DC z enkoderami) upraszcza sterowanie prędkością — prędkość zależy wprost od częstotliwości kroków, bez potrzeby pomiaru obrotów.
- Sprzętowe generowanie kroków w przerwaniu Timer1 zapewnia stały, precyzyjny rytm impulsów niezależnie od obciążenia głównej pętli.
- **Filtr komplementarny** okazał się wystarczający i znacznie prostszy obliczeniowo niż filtr Kalmana — adekwatny do mocy ATmega328P.
- Człon różniczkujący liczony z pomiaru (żyroskopu) eliminuje gwałtowne reakcje regulatora i poprawia stabilność.

Napotkane problemy

- Wcześniejszy prototyp na innych sterownikach (TMC2209) nie pracował poprawnie i został zastąpiony obecnym rozwiązaniem (TMC2226).
- Przy zasilaniu z pakietu akumulatorów 3S robot dotąd nie pracuje stabilnie (zob. sekcja 3.4) — do testów używany jest konwerter USB-UART; docelowo planowane jest dedykowane zasilanie 9 V.
- Na twardych, gładkich nawierzchniach widoczne są większe drgania (poślizg kół) — por. wykres telemetry w rozdz. 8.
- **Zawieszanie magistrali I²C po upadku** — po przewróceniu czujnik MPU-6050 potrafił „zablokować” magistralę (przytrzymanie linii SDA), przez co sam reset nie wystarczał i trzeba było ponownie wgrywać program. Rozwiązano przez procedurę odblokowania magistrali (wyklocowanie linii SCL) oraz programowy reset czujnika przy każdym starcie — teraz zwykły reset przywraca działanie.
- Dobór punktu równowagi (TARGET_ANGLE) po zmianie masy (np. dołożeniu baterii) — wyznaczany z monitora szeregowego (zob. procedurę uruchomienia oraz rozdz. 8).

Ograniczenia obecnej wersji

- W wersji podstawowej oba silniki kroczą z tą samą częstotliwością — robot balansuje i jedzie prosto, lecz nie skręca. Opracowano jednak wariant z automatycznym obrotem (sekcja 6.6), w którym robot co ~13 s wykonuje obrót w lewo o 45° — wariant ten wymaga jeszcze weryfikacji na sprężenie.
- Brak zdalnego sterowania (jazda do przodu/tyłu, obrót na żądanie).
- Połączenia wykonano punkt-punkt na płytce uniwersalnej; schemat ideowy przygotowano (sekcja 3.6, dodatek z plikiem KiCad).

10. Kierunki dalszego rozwoju

- **Sterowanie kierunkiem** — dodanie różnicy prędkości między kołami (osobny okres kroku per silnik) umożliwi skręcanie i jazdę zadaną.
- **Zdalne sterowanie** — moduł Bluetooth (HC-05) lub nRF24L01 do zadawania ruchu i zmiany nastaw w locie.
- **Sterownik prędkości i pozycji** — zewnętrzna pętla regulująca prędkość/położenie (kaskada), aby robot nie „odjeżdżał”.
- **Projekt dedykowanego PCB** — przeniesienie układu z perfboardu na płytkę drukowaną (zwiększenie niezawodności).
- Telemetria bezprzewodowa i logowanie danych do analizy strojenia.
- **Wykorzystanie UART sterowników TMC2226** — konfiguracja prądu i mikrokroku programowo, diagnostyka StallGuard.

11. Bibliografia i materiały źródłowe

Karty katalogowe i literatura

- [1] Microchip Technology, ATmega48A/PA/88A/PA/168A/PA/328/P Data Sheet (rdzeń AVR, Timer1, TWI/I²C, tryb CTC).
- [2] InvenSense (TDK), MPU-6000/MPU-6050 Product Specification oraz Register Map and Descriptions, rev. 4.2.
- [3] Analog Devices/Trinamic, TMC2226 Datasheet (StealthChop, tryb STEP/DIR, tabela mikrokroku MS1/MS2); BIGTREETECH, TMC2226 V1.0 — dokumentacja modułu.
- [4] Monolithic Power Systems, MP1584 — 3A, 1.5 MHz, 28 V Step-Down Converter.
- [5] Silicon Labs, CP2102/9 — USB to UART Bridge Controller.
- [6] Karta katalogowa silnika krokowego JK42HS40-1704 (NEMA 17, 200 kroków/obr., 1,8°, 2,8 V / 1,7 A, 0,4 N·m).
- [7] K. J. Åström, T. Hägglund, PID Controllers: Theory, Design, and Tuning, ISA, wyd. 2.
- [8] R. P. M. Chan i in., Review of modelling and control of two-wheeled robots, Annual Reviews in Control, 2013.
- [9] Poradniki budowy robotów samobalansujących (Instructables, Circuit.rocks, Hackster) — metoda wyznaczania kąta równowagi z monitora szeregowego oraz konieczność ponownego strojenia PID po zmianie masy/rozkładu ciężaru.

Źródła komponentów (sklepy)

Mikrokontroler: AVR - ATmega328P-U DIP

<https://botland.com.pl/avr-w-obudowie-tht/1264-mikrokontroler-avr-atmega328p-u-dip-5903351249928.html>

Silniki Krokowe x2 - JK42HS40-1704 200 kroków/obr 2,8V / 1,7A / 0,4Nm

<https://botland.com.pl/silniki-krokowe/6429-silnik-krokowy-jk42hs40-1704-200-krowowobr-28v17a04nm-5904422334635.html>

Stepsticki x2 - RepRap 35V/2A Pololu 1182

<https://botland.com.pl/stepstick-sterowniki-do-silnikow-krowowych/2964-sterownik-silnika-krowowego-a4988-reprap-zielony-5904422331351.html>

Huby montażowe - HEX 12mm 5mm wał

https://allegro.pl/oferta/4-sztuk-5mm-do-12mm-adapter-szesciokatny-rc-metalowe-kolo-hex-hub-adapter-rc-car-czarny-14407779434?reco_id=c36e6716-27a7-11f1-b14c-9ac87c8b4eba&sid=1147bbb04b955b96a226cab2e664258d9124ef0862320b2d7366fdd6604cf017

Koła - RC1:10 na HEX 12mm

<https://allegro.pl/oferta/4-sztuk-rc-1-10-na-oponach-szosowych-kolo-rajdowe-1-9inch-hex-12mm-dla-cc01-czarny-15736425622>

Tuleje dystansowe mosiężne - 25mm

<https://botland.com.pl/tuleje-dystansowe/7803-tuleja-dystansowa-mosieczna-25mm-10szt-5904422308155.html>

Mocowanie L NEMA 17

<https://botland.com.pl/mocowania-silnikow/6286-mocowanie-stalowe-typu-l-do-silnika-krowowego-nema-17-5904422334512.html>

Pleksa

<https://botland.com.pl/plyty-akrylowe-plexi/20906-plexi-przezroczysta-wylewana-3mm-200x200mm-5szt-5907255402885.html>

3-osiowy żyroskop i akcelerometr

<https://botland.com.pl/zyroskopy/3888-mpu-6050-3-osiowy-akcelerometr-i-zyroskop-i2c-modul-dfrobot-6959420908240.html>

Akumulatory x3 - 18650 Li-Ion INR18650-F1HR 3350mAh

<https://botland.com.pl/akumulatory-li-ion/22842-ogniwo-18650-li-ion-inr18650-f1hr-3350mah.html>

Koszyk na 3 akumulatory

<https://botland.com.pl/koszyki-na-baterie/5240-koszyk-na-2-akumulatory-typu-18650-polaczenie-szeregowe-5904422374341.html>

Stabilizator napięcia 5V - L7805CV THT

<https://botland.com.pl/regulatory-napiecia/3092-stabilizator-5v-l7805cv-tht-to220-5szt-5904422307219.html>

Kwarc 16MHz - HC49

<https://botland.com.pl/rezonatory/166-rezonator-kwarcowy-16mhz-hc49-niski-10szt-5904422358761.html>

Podstawka do układów DIP 28 pin

<https://botland.com.pl/podstawki-pod-uklady-scalone/90-podstawka-do-ukladow-dip-28-pin-waska-5szt-5904422355326.html>

Konwerter USB-UART

<https://botland.com.pl/konwertery-usb-uart-rs232-rs485/4924-konwerter-usb-uart-cp2102-wtyk-usb-waveshare-11079-5904422374204.html>

Przewody połączeniowe (męsko-męskie, męsko-żeńskie, żeńsko-żeńskie)

<https://botland.com.pl/przewody-polaczeniowe-mesko-meskie/19947-przewody-polaczeniowe-mesko-meskie-justpi-10cm-40szt-5904422328689.html>

<https://botland.com.pl/przewody-polaczeniowe-zensko-zenskie/19948-przewody-polaczeniowe-zensko-zenskie-justpi-10cm-40szt-5904422328672.html>

<https://botland.com.pl/przewody-polaczeniowe-zensko-meskie/19949-przewody-polaczeniowe-zensko-meskie-justpi-10cm-40szt-5904422328696.html>

UPDATE

Sterownik do silnika krokowego x2 - BIGTREETECH TMC2226

<https://allegro.pl/oferta/bigtreotech-tmc2226-v1-0-sterownik-silnika-krokowego-tmc2226-16225756502>

Przetwornica Step-Down DC-DC MP1584EN MAX 3A 5V

<https://allegro.pl/oferta/przetwornica-dc-dc-mp1584en-max-3a-5v-micro-2g-step-down-do-fpv-18234233285>

Dodatek A. Pełny listing oprogramowania

Poniżej zamieszczono kompletny kod źródłowy firmware sterującego robotem (plik balanser.ino).

```

1 #include <Wire.h>
2
3 #define STEP_LEFT    6    // PD6
4 #define DIR_LEFT     7    // PD7
5 #define STEP_RIGHT   8    // PB0
6 #define DIR_RIGHT    9    // PB1
7
8 #define STEP_L_BIT    (1 << PD6)
9 #define STEP_R_BIT    (1 << PB0)
10 #define DIR_L_BIT     (1 << PD7)
11 #define DIR_R_BIT     (1 << PB1)
12
13 float Kp = 65.0;
14 float Ki = 0.4;
15 float Kd = 1.0;
16
17 float TARGET_ANGLE = 5.6;
18
19 #define LOOP_HZ        250
20 #define LOOP_US        (1000000UL / LOOP_HZ)
21
22 #define COMP_ALPHA      0.98f
23
24 #define D_LPF_ALPHA     0.30f
25
26 #define MAX_SPEED       2500.0f
27 #define MAX_ACCEL       60000.0f
28 #define SPEED_TO_DELAY_K 100000.0f
29 #define MIN_STEP_PERIOD_US 40
30 #define MAX_STEP_PERIOD_US 20000
31 #define I_LIMIT         25.0f
32
33 #define FALL_LIMIT      35.0f
34
35 #define AUTO_TRIM_GAIN  0.0f
36 #define TRIM_LIMIT      7.0f
37
38 #define DEBUG_SERIAL    1
39 #define DEBUG_EVERY_N   25                // co ile petli drukowac
40
41 float compAngle = 0.0f;
42 float gyroBiasX = 0.0f;
43 float errorIntegral = 0.0f;
44 float dTermFilt = 0.0f;
45 float cmdSpeed = 0.0f;    // aktualna (wygladzona) predkosć komendy
46 float angleTrim = 0.0f;  // korekta setpointu z auto-trim
47
48 volatile bool motorsRunning = false;
49
50 unsigned long lastLoopUs = 0;
51 uint16_t dbgCounter = 0;
52
53 ISR(TIM1_COMP1_vect) {
54     if (motorsRunning) {
55         PORTD |= STEP_L_BIT;
56         PORTB |= STEP_R_BIT;
57         // impuls > 100 ns (wymog TMC2226)
58         asm volatile("nop\n\tNOP\n\tNOP\n\tNOP\n\tNOP\n\tNOP\n\tNOP\n\t");
59         PORTD &= ~STEP_L_BIT;
60         PORTB &= ~STEP_R_BIT;
61     }
62 }
63
64 void mpuWrite(uint8_t reg, uint8_t val) {
65     Wire.beginTransmission(0x68);
66     Wire.write(reg);

```

```

67   Wire.write(val);
68   Wire.endTransmission();
69 }
70
71 void mpuInit() {
72   mpuWrite(0x6B, 0x00); // PWR_MGMT_1 - wybudz
73   delay(10);
74   mpuWrite(0x1A, 0x03); // CONFIG/DLPF=3 -> zyro 42 Hz / akcel 44 Hz (kluczowe!)
75   mpuWrite(0x1B, 0x00); // GYRO_CONFIG -> +-250 st/s (131 LSB/(st/s))
76   mpuWrite(0x1C, 0x00); // ACCEL_CONFIG -> +-2 g
77   delay(10);
78 }
79
80 void calibrateGyro() {
81   long sum = 0;
82   for (int i = 0; i < 1000; i++) { // ~2 s - TRZYMAJ ROBOTA NIERUCHOMO
83     Wire.beginTransaction(0x68);
84     Wire.write(0x43); // GYRO_XOUT_H
85     Wire.endTransmission();
86     Wire.requestFrom(0x68, 2);
87     sum += (int16_t)(Wire.read() << 8 | Wire.read());
88     delay(2);
89   }
90   gyroBiasX = sum / 1000.0f;
91 }
92
93 void setup() {
94   Wire.begin();
95   Wire.setClock(400000);
96   Serial.begin(115200);
97
98   pinMode(DIR_LEFT, OUTPUT);
99   pinMode(STEP_LEFT, OUTPUT);
100  pinMode(DIR_RIGHT, OUTPUT);
101  pinMode(STEP_RIGHT, OUTPUT);
102
103  // Timer1: CTC, prescaler 8 (2 MHz -> 0.5 us/tik)
104  cli();
105  TCCR1A = 0;
106  TCCR1B = 0;
107  TCNT1 = 0;
108  OCR1A = 65535;
109  TCCR1B |= (1 << WGM12); // CTC
110  TCCR1B |= (1 << CS11); // prescaler 8
111  TIMSK1 |= (1 << OCIE1A); // przerwanie przy zrownaniu
112  sei();
113
114  mpuInit();
115  delay(100);
116  calibrateGyro();
117
118  Wire.beginTransaction(0x68);
119  Wire.write(0x3B);
120  Wire.endTransmission();
121  Wire.requestFrom(0x68, 6);
122  int16_t ax = Wire.read() << 8 | Wire.read(); (void)ax;
123  int16_t ay = Wire.read() << 8 | Wire.read();
124  int16_t az = Wire.read() << 8 | Wire.read();
125  compAngle = atan2((float)ay, (float)az) * 180.0f / PI;
126
127  lastLoopUs = micros();
128 }
129
130 void loop() {
131   unsigned long now = micros();
132   if ((now - lastLoopUs) < LOOP_US) return; // czekaj na slot petli
133   float dt = (now - lastLoopUs) * 1e-6f;
134   lastLoopUs = now;
135
136   // --- Odczyt IMU (burst 14 bajtow jednym strzałem) ---
137   Wire.beginTransaction(0x68);
138   Wire.write(0x3B);
139   Wire.endTransmission();

```

```

140 Wire.requestFrom(0x68, 14);
141 int16_t accX = Wire.read() << 8 | Wire.read(); (void)accX;
142 int16_t accY = Wire.read() << 8 | Wire.read();
143 int16_t accZ = Wire.read() << 8 | Wire.read();
144 Wire.read(); Wire.read(); // temperatura - pomijamy
145 int16_t gyroXr = Wire.read() << 8 | Wire.read();
146 Wire.read(); Wire.read(); // gyroY - pomijamy
147 Wire.read(); Wire.read(); // gyroZ - pomijamy
148
149 float gyroX = (gyroXr - gyroBiasX) / 131.0f; // [st/s]
150
151 // --- Filtr komplementarny ---
152 float accAngle = atan2((float)accY, (float)accZ) * 180.0f / PI;
153 compAngle = COMP_ALPHA * (compAngle + gyroX * dt) + (1.0f - COMP_ALPHA) * accAngle;
154
155 // --- Setpoint (z ewentualna korekta auto-trim) i blad ---
156 float setpoint = TARGET_ANGLE + angleTrim;
157 float error = setpoint - compAngle;
158
159 // --- Bezpieczenstwo: upadek -> stop i reset integratorow ---
160 if (fabs(error) > FALL_LIMIT) {
161     motorsRunning = false;
162     errorIntegral = 0.0f;
163     cmdSpeed = 0.0f;
164     // angleTrim zostawiamy - to wyznaczony punkt rownowagi
165     return;
166 }
167
168 // --- PID ---
169 errorIntegral += error * dt;
170 errorIntegral = constrain(errorIntegral, -I_LIMIT, I_LIMIT);
171
172 // czlon D na pomiarze (predkosc katowa), filtrowany dolnoprzepustowo
173 float dRaw = -gyroX;
174 dTermFilt += D_LPF_ALPHA * (dRaw - dTermFilt);
175
176 float output = Kp * error + Ki * errorIntegral + Kd * dTermFilt;
177
178 // --- Wygladzenie komendy predkosci (slew-rate limiter) ---
179 output = constrain(output, -MAX_SPEED, MAX_SPEED);
180 float maxDelta = MAX_ACCEL * dt;
181 float delta = constrain(output - cmdSpeed, -maxDelta, maxDelta);
182 cmdSpeed += delta;
183
184 // --- Auto-trim punktu rownowagi (wolna petla, opcjonalna) ---
185 if (AUTO_TRIM_GAIN > 0.0f) {
186     angleTrim += AUTO_TRIM_GAIN * cmdSpeed * dt;
187     angleTrim = constrain(angleTrim, -TRIM_LIMIT, TRIM_LIMIT);
188 }
189
190 // --- Wysluj do silnikow ---
191 setMotorSpeed(cmdSpeed);
192
193 // --- Debug ---
194 #if DEBUG_SERIAL
195 if (++dbgCounter >= DEBUG_EVERY_N) {
196     dbgCounter = 0;
197     Serial.print("ang="); Serial.print(compAngle, 2);
198     Serial.print(" err="); Serial.print(error, 2);
199     Serial.print(" out="); Serial.print(output, 0);
200     Serial.print(" spd="); Serial.print(cmdSpeed, 0);
201     Serial.print(" trim="); Serial.println(angleTrim, 2);
202 }
203 #endif
204 }
205
206 void setMotorSpeed(float speed) {
207     // kierunek (DIR jednego silnika odwrocony - lustrzane zamontowanie)
208     if (speed >= 0) {
209         PORTD |= DIR_L_BIT;
210         PORTB &= ~DIR_R_BIT;
211     } else {
212         PORTD &= ~DIR_L_BIT;

```

```

213     PORTB |= DIR_R_BIT;
214 }
215
216 float mag = fabs(speed);
217
218 if (mag < 1.0f) {           // praktycznie zero -> trzymaj pradem, nie krokuj
219     motorsRunning = false;
220     return;
221 }
222
223 // predkosc -> okres kroku [us]
224 float periodUs = SPEED_TO_DELAY_K / mag;
225 if (periodUs < MIN_STEP_PERIOD_US) periodUs = MIN_STEP_PERIOD_US;
226 if (periodUs > MAX_STEP_PERIOD_US) periodUs = MAX_STEP_PERIOD_US;
227
228 // okres[us] -> tiki timera (0.5 us/tik => *2)
229 uint16_t newOCR = (uint16_t)(periodUs * 2.0f);
230
231 cli();
232 OCR1A = newOCR;
233 if (TCNT1 > newOCR) TCNT1 = 0; // unik glitcha zawinięcia licznika przy przyspieszaniu
234 sei();
235
236 motorsRunning = true;
237 }

```

Dodatek B. Listing wariantu z obrotem

Poniżej kompletny kod wariantu z automatycznym obrotem co ~13 s (plik balanser_obrot.ino; opis w sekcji „6.6. Rozszerzenie: automatyczny obrót”). Wariant eksperymentalny — przed użyciem należy zweryfikować kierunek obrotu i dobrać parametry na sprzęcie.

```
1 // =====
2 // Robot samobalansujący - WARIANT Z AUTOMATYCZNYM OBROTEM (v2)
3 // Co TURN_INTERVAL_MS robot wykonuje obrót w lewo o TURN_ANGLE_DEG,
4 // nie przerywając balansowania.
5 //
6 // Krokowanie obu silników: jedno LEKKIE przerwanie Timer1 o stałej,
7 // UMIARKOWANEJ częstotliwości (STEP_ISR_HZ) z dwoma akumulatorami fazy
8 // (DDS). Arytmetyka 16-bitowa, kierunek ustawiany POZA przerwaniem -
9 // dzięki czemu ISR jest bardzo krótki i nie zagłusza petli/I2C.
10 // Prędkość każdego koła = składowa balansu +/- składowa obrotu.
11 //
12 // UWAGA: w razie braku obrotu/złej strony - zmienić znak TURN_DIR;
13 // prędkość obrotu reguluje TURN_RATE. Płyta w Arduino IDE: "Arduino Uno"
14 // (ATmega328P, 16 MHz).
15 // =====
16 #include <Wire.h>
17
18 #define STEP_LEFT 6 // PD6
19 #define DIR_LEFT 7 // PD7
20 #define STEP_RIGHT 8 // PB0
21 #define DIR_RIGHT 9 // PB1
22
23 #define STEP_L_BIT (1 << PD6)
24 #define STEP_R_BIT (1 << PB0)
25 #define DIR_L_BIT (1 << PD7)
26 #define DIR_R_BIT (1 << PB1)
27
28 // --- regulator (jak w wersji podstawowej) ---
29 float Kp = 65.0;
30 float Ki = 0.4;
31 float Kd = 1.0;
32 float TARGET_ANGLE = 5.6; // bazowy kąt równowagi (może być nadpisany auto-kalibracją)
33 float angleTrim = 0.0f; // powolna korekta setpointu (auto-trim)
34
35 // === AUTO-KALIBRACJA KĄTA RÓWNOWAGI ===
36 // Punkt równowagi zależy od rozkładu masy (np. bateria 9V na gorze go przesuwają), więc
37 // sztywne 5,6 stopnia bywa złe -> robot ma stałe przechylenie i "odjeżdża".
38 // AUTO_TARGET_AT_START: po kalibracji żyroskopu przyjmij kąt, pod którym TRZYMANO robota,
39 // jako punkt równowagi. -> Trzymaj robota DOKŁADNIE w pionie/równowadze przez te ~2 s.
40 // AUTO_TRIM: powolne dostrajanie setpointu w trakcie pracy, by średnia prędkość koła = 0
41 // (robot przestaje "odjeżdżać"). UWAGA: gdyby zaczął narastająco odjeżdżać - zmień znak.
42 // WAŻNE: AUTO_TARGET domyślnie WYLACZONE. Przechwytywało kąt, pod którym TRZYMANO robota,
43 // a to rzadko jest dokładny punkt równowagi -> robot nie balansował i się wywracał.
44 // PEWNY sposób ustawienia równowagi (recznie): postaw/przytrzymaj robota dokładnie w
45 // równowadze, odczytaj 'ang=' z monitora szeregowego (115200) i wpisz tę wartość do
46 // TARGET_ANGLE powyżej. Dopiero gdy to działa - można ewentualnie spróbować AUTO_TARGET=1
47 // (trzymając robota PRECYZYJNIE w punkcie równowagi przez czas kalibracji).
48 #define AUTO_TARGET_AT_START 0 // opcjonalne, domyślnie WYL
49 #define AUTO_TRIM 0 // opcjonalne, domyślnie WYL (zły znak -> narastający dryf)
50 #define AUTO_TRIM_GAIN 0.0006f // małe! dostrajanie jest wolne
51 #define TRIM_LIMIT 8.0f // ograniczenie korekty [stopnie]
52
53 #define LOOP_HZ 250
54 #define LOOP_US (1000000UL / LOOP_HZ)
55 #define COMP_ALPHA 0.98f
56 #define D_LPF_ALPHA 0.30f
57 #define DEBUG_SERIAL 1 // 0 = wyłącz telemetrię (zero narzutu na float->tekst)
58 #define DEBUG_EVERY_N 25 // co ile petli drukować (25 => ~10 Hz)
59
60 // Budżet CPU (ATmega328P @16 MHz): ISR krokowania 25 kHz, lekki (16-bit) ~ 8%;
61 // odczyt I2C 14 bajtów @400 kHz ~ 10%; obliczenia petli 250 Hz (1x atan2 + float) < 10%.
62 // Zapas > 70% - procesor nie jest obciążony. (Telemetria float jest najcięższą częścią)
63 // opcjonalna - można ją wyłączyć DEBUG_SERIAL=0.)
64
```

```

65 #define MAX_SPEED          2500.0f    // jak w oryginale -> rate <= 25000
66 #define MAX_ACCEL          60000.0f
67 #define SPEED_TO_RATE      10.0f      // cmdSpeed -> mikrokroki/s
68 #define MAX_RATE           25000     // ograniczenie czestotliwosci krokow [1/s]
69 #define RATE_DEADBAND      8          // ponizej tej predkosci nie krokuje (jak oryginal)
70 #define I_LIMIT            25.0f
71 #define FALL_LIMIT         35.0f
72
73 // --- krokowanie DDS (lekki ISR) ---
74 // Timer1 CTC, presk. 8 (2 MHz) -> OCR1A = 79 -> 25 000 Hz
75 #define STEP_ISR_HZ        250000
76 volatile uint16_t rateMagL = 0;       // |predkosc| lewego [mikrokroki/s]
77 volatile uint16_t rateMagR = 0;       // |predkosc| prawego
78 volatile uint16_t accL = 0, accR = 0; // akumulatory fazy
79 volatile bool      motorsEnabled = false;
80
81 // --- parametry obrotu ---
82 #define MICROSTEPS_PER_REV 6400.0f    // 200 krokow * 1/32 mikrokroku
83 #define WHEEL_DIAM_CM      7.0f
84 #define TRACK_CM           15.5f      // rozstaw kol
85 #define TURN_ANGLE_DEG     45.0f      // mniejszy obrot = mniejsze zaburzenie, latwiej utrzymac pion
86 #define TURN_INTERVAL_MS   13333UL    // ~co 13 s (1,5x czesciej niz 20 s)
87 #define TURN_RATE          1200.0f    // roznicowa predkosc obrotu [mikrokroki/s] (wolniej = pewniej)
88 #define TURN_RAMP           3000.0f    // przyspieszenie obrotu [mikrokroki/s^2] (lagodny start)
89 #define TURN_DIR            (+1.0f)    // +1 obrot w lewo (ew. zmienic na -1)
90 // PRIORYTET BALANSU - PROPORCJONALNY (zamiast binarnego, ktory blokowal obrot na stale):
91 // |error| < TURN_PITCH_FULL -> pelna predkosc obrotu,
92 // FULL..STOP -> predkosc obrotu LINIOWO maleje do 0 (przy chwilowym
93 // przechyle obrot tylko zwalnia i mimo to sie konczy),
94 // |error| >= TURN_PITCH_EMERG -> NATYCHMIASTOWE odciecie (awaryjne, duzy przechyl).
95 // Mniejsze progi = obrot zwalnia juz przy malym przechyle -> robot trzyma sie blizej PIONU
96 // podczas obrotu (mniej "wahania"), kosztem dluzszego czasu obrotu.
97 #define TURN_PITCH_FULL    1.5f       // [stopnie] ponizej -> pelny obrot
98 #define TURN_PITCH_STOP    4.0f       // [stopnie] przy tym przechyle predkosc obrotu = 0
99 #define TURN_PITCH_EMERG   7.0f       // [stopnie] awaryjne natychmiastowe odciecie
100 // PRZYGOTOWANIE DO OBROTU: po uplywie interwalu obrot jest "uzbrajany", ale startuje
101 // dopiero, gdy robot jest pionowo i spokojny przez TURN_READY_TIME (wejście w obrot
102 // z dobrego stanu = duzo pewniej, zwlaszcza przy ciezkiej gorze).
103 #define TURN_READY_ANGLE   1.5f       // |error| ponizej -> uznaj za pionowo [stopnie]
104 #define TURN_READY_RATE    40.0f      // |predkosc katowa pochylenia| ponizej [stopnie/s]
105 #define TURN_READY_SPEED   120.0f     // |cmdSpeed| ponizej -> robot nie jedzie/spokojny
106 #define TURN_READY_TIME    0.30f      // wymagany czas ciaglej stabilnosci [s]
107 #define TURN_ARM_TIMEOUT_MS 4000UL    // gdy nie wyciszy sie w tym czasie - obrot mimo to
108
109 // --- opcjonalne utrzymanie kursu (yaw) = rozna predkosc kol w petli balansu ---
110 // Wykorzystuje zyroskop w osi Z (dotad pomijany). Korekta jest ROZNICOWA, wiec nie
111 // odbiera autorytetu balansowi (wspolnemu). Efekt: robot nie znosi/nie obraca sie sam,
112 // trzyma kurs i opiera sie probie skrecenia. 1 = wlacz, 0 = wylacz.
113 #define ENABLE_YAW_HOLD    0
114 #define KYAW_P             40.0f      // wzmacnienie od bledu kursu [stopnie]
115 #define KYAW_D             8.0f       // wzmacnienie od predkosci katowej yaw [stopnie/s]
116 #define YAW_CORR_LIMIT     1500.0f   // ograniczenie korekty roznicowej [mikrokroki/s]
117
118 // --- opcjonalne sterowanie pozycja / jazda przod-tyl ---
119 // Silnik krokowy = znana predkosc, wiec calka cmdSpeed daje pozycje (darmowa odometria).
120 // Petla ZEWNETRZNA lekko pochyła setpoint, by trzymac pozycje (anti-drift, driveCmd=0)
121 // albo jechac z zadana predkoscia (driveCmd != 0). Jazda na balansie = sterowanie
122 // pochyleniem, NIE bezposrednio predkoscia kol. 1 = wlacz, 0 = wylacz.
123 #define ENABLE_POS_HOLD    0
124 #define KPOS_P             0.0006f    // od bledu pozycji [stopnie/krok]
125 #define KPOS_D             0.004f     // od predkosci [stopnie/(krok/s)]
126 #define SETPOINT_OFFSET_LIMIT 3.0f    // max pochylenie zadane przez petle pozycji [stopnie]
127 float driveCmd = 0.0f;               // zadana predkosc jazdy [mikrokroki/s]; 0=stoj, >0=przod
128 float posEst = 0.0f;                 // estymata pozycji [mikrokroki]
129
130 float compAngle = 0.0f;
131 float gyroBiasX = 0.0f;
132 float gyroBiasZ = 0.0f; // offset zyroskopu osi Z (yaw)
133 float heading = 0.0f; // scalony kurs [stopnie]
134 float errorIntegral = 0.0f;
135 float dTermFilt = 0.0f;
136 float cmdSpeed = 0.0f;
137

```

```

138 unsigned long lastLoopUs = 0;
139 uint32_t      lastTurnMs = 0;
140 float         turnStepsLeft = 0.0f;
141 float         turnCmd = 0.0f;          // biezaca (wygladzona) predkosc obrotu
142 bool          turning = false;
143 bool          turnArmed = false;       // faza przygotowania - czeka na wyciszenie
144 float         settleTimer = 0.0f;     // czas ciaglej stabilnosci [s]
145 uint32_t      armMs = 0;              // moment uzbrojenia (do timeoutu)
146 uint16_t      dbgCounter = 0;
147
148 // liczba mikrokrokow na kolo dla obrotu o TURN_ANGLE_DEG:
149 const float TURN_STEPS =
150     (TURN_ANGLE_DEG / 180.0f) * (TRACK_CM * 0.5f) / WHEEL_DIAM_CM * MICROSTEPS_PER_REV;
151
152 // --- ISR krokowania: krotki, 16-bitowy, bez galezi na znak ---
153 ISR(TIMER1_COMPA_vect) {
154     if (!motorsEnabled) return;
155     accL += rateMagL;
156     if (accL >= STEP_ISR_HZ) {
157         accL -= STEP_ISR_HZ;
158         PORTD |= STEP_L_BIT;
159         asm volatile("nop\n\tnop\n\tnop\n\tnop\n\tnop\n\tnop\n\tnop\n\t");
160         PORTD &= ~STEP_L_BIT;
161     }
162     accR += rateMagR;
163     if (accR >= STEP_ISR_HZ) {
164         accR -= STEP_ISR_HZ;
165         PORTB |= STEP_R_BIT;
166         asm volatile("nop\n\tnop\n\tnop\n\tnop\n\tnop\n\tnop\n\tnop\n\t");
167         PORTB &= ~STEP_R_BIT;
168     }
169 }
170
171 // Odblokowanie ZAWIESZONEJ magistrali I2C (po upadku/zaniku zasilania MPU czujnik
172 // potrafi trzymac SDA nisko; sam reset MCU tego nie naprawia). Wyklucz do 9 impulsow
173 // SCL, az slave zwolni SDA, potem wygeneruj warunek STOP. SDA=A4, SCL=A5.
174 void i2cRecover() {
175     pinMode(A4, INPUT_PULLUP); // SDA
176     pinMode(A5, INPUT_PULLUP); // SCL
177     delayMicroseconds(10);
178     for (uint8_t i = 0; i < 9 && digitalRead(A4) == LOW; i++) {
179         pinMode(A5, OUTPUT); digitalWrite(A5, LOW); delayMicroseconds(5); // SCL low
180         pinMode(A5, INPUT_PULLUP); delayMicroseconds(5); // SCL high (pullup)
181     }
182     // warunek STOP: SDA z LOW na HIGH przy SCL HIGH
183     pinMode(A4, OUTPUT); digitalWrite(A4, LOW); delayMicroseconds(5);
184     pinMode(A5, INPUT_PULLUP); delayMicroseconds(5);
185     pinMode(A4, INPUT_PULLUP); delayMicroseconds(5);
186 }
187
188 void mpuWrite(uint8_t reg, uint8_t val) {
189     Wire.beginTransmission(0x68);
190     Wire.write(reg);
191     Wire.write(val);
192     Wire.endTransmission();
193 }
194
195 void mpuInit() {
196     mpuWrite(0x6B, 0x80); // DEVICE_RESET - przywroc czujnik do stanu znanego (wazne po upadku)
197     delay(100);
198     mpuWrite(0x6B, 0x00); // PWR_MGMT_1 - wybudz (skasuj sleep)
199     delay(10);
200     mpuWrite(0x1A, 0x03); // DLPF=3 -> zyro 42 Hz
201     mpuWrite(0x1B, 0x00); // GYRO_CONFIG +-250 st/s (131 LSB/(st/s))
202     mpuWrite(0x1C, 0x00); // ACCEL_CONFIG +-2 g
203     delay(10);
204 }
205
206 void calibrateGyro() {
207     long sumX = 0, sumZ = 0;
208     for (int i = 0; i < 1000; i++) { // ~2 s - TRZYMAJ ROBOTA NIERUCHOMO
209         Wire.beginTransmission(0x68);
210         Wire.write(0x43); // GYRO_XOUT_H .. ZOUT_L

```

```

211     Wire.endTransmission();
212     Wire.requestFrom(0x68, 6);
213     int16_t gx = Wire.read() << 8 | Wire.read();
214     Wire.read(); Wire.read(); // gyroY - pomijamy
215     int16_t gz = Wire.read() << 8 | Wire.read();
216     sumX += gx; sumZ += gz;
217     delay(2);
218 }
219 gyroBiasX = sumX / 1000.0f;
220 gyroBiasZ = sumZ / 1000.0f;
221 }
222
223 // ustaw kierunki (DIR) poza ISR i zapisz magnitudy predkosci kol
224 void setMotorRates(float rL, float rR) {
225     if (rL >= 0) PORTD |= DIR_L_BIT; else PORTD &= ~DIR_L_BIT;
226     if (rR >= 0) PORTB &= ~DIR_R_BIT; else PORTB |= DIR_R_BIT; // prawy odwrocony
227
228     float mL = fabs(rL); if (mL > MAX_RATE) mL = MAX_RATE;
229     float mR = fabs(rR); if (mR > MAX_RATE) mR = MAX_RATE;
230     if (mL < RATE_DEADBAND) mL = 0; // brak chaotycznego krokowania przy ~zerowej predkosci
231     if (mR < RATE_DEADBAND) mR = 0;
232
233     uint16_t iL = (uint16_t)mL;
234     uint16_t iR = (uint16_t)mR;
235     cli();
236     rateMagL = iL;
237     rateMagR = iR;
238     sei();
239 }
240
241 void setup() {
242     i2cRecover(); // odblokuj ewentualnie zawieszona magistrale I2C (po upadku)
243     Wire.begin();
244     Wire.setClock(400000);
245     Serial.begin(115200);
246
247     pinMode(DIR_LEFT, OUTPUT);
248     pinMode(STEP_LEFT, OUTPUT);
249     pinMode(DIR_RIGHT, OUTPUT);
250     pinMode(STEP_RIGHT, OUTPUT);
251
252     // Timer1: CTC, presk. 8 (2 MHz). OCR1A=79 -> (79+1) tikow -> 25 000 Hz
253     cli();
254     TCCR1A = 0;
255     TCCR1B = 0;
256     TCNT1 = 0;
257     OCR1A = 79;
258     TCCR1B |= (1 << WGM12); // CTC
259     TCCR1B |= (1 << CS11); // presk. 8
260     TIMSK1 |= (1 << OCIE1A);
261     sei();
262
263     mpuInit();
264     delay(100);
265     calibrateGyro();
266
267     Wire.beginTransmission(0x68);
268     Wire.write(0x3B);
269     Wire.endTransmission();
270     Wire.requestFrom(0x68, 6);
271     int16_t ax = Wire.read() << 8 | Wire.read(); (void)ax;
272     int16_t ay = Wire.read() << 8 | Wire.read();
273     int16_t az = Wire.read() << 8 | Wire.read();
274     compAngle = atan2((float)ay, (float)az) * 180.0f / PI;
275
276     #if AUTO_TARGET_AT_START
277     TARGET_ANGLE = compAngle; // kat, pod którym trzymano robota podczas kalibracji = rownowaga
278     #endif
279
280     lastLoopUs = micros();
281     lastTurnMs = millis();
282 }
283

```

```

284 void loop() {
285     unsigned long now = micros();
286     if ((now - lastLoopUs) < LOOP_US) return;
287     float dt = (now - lastLoopUs) * 1e-6f;
288     lastLoopUs = now;
289
290     // --- Odczyt IMU (burst 14 bajtów) ---
291     Wire.beginTransmission(0x68);
292     Wire.write(0x3B);
293     Wire.endTransmission();
294     Wire.requestFrom(0x68, 14);
295     int16_t accX = Wire.read() << 8 | Wire.read(); (void)accX;
296     int16_t accY = Wire.read() << 8 | Wire.read();
297     int16_t accZ = Wire.read() << 8 | Wire.read();
298     Wire.read(); Wire.read(); // temperatura
299     int16_t gyroXr = Wire.read() << 8 | Wire.read();
300     Wire.read(); Wire.read(); // gyroY - pomijamy
301     int16_t gyroZr = Wire.read() << 8 | Wire.read(); // gyroZ - kurs (yaw)
302
303     float gyroX = (gyroXr - gyroBiasX) / 131.0f;
304     float yawRate = (gyroZr - gyroBiasZ) / 131.0f; // [stopnie/s]
305
306     // --- Filtr komplementarny ---
307     float accAngle = atan2((float)accY, (float)accZ) * 180.0f / PI;
308     compAngle = COMP_ALPHA * (compAngle + gyroX * dt) + (1.0f - COMP_ALPHA) * accAngle;
309
310     // --- Pętla zewnętrzna: utrzymanie pozycji / jazda przód-tył (opcjonalna) ---
311     // Aktywna tylko gdy włączona globalnie (ENABLE_POS_HOLD). NIE jest włączana podczas
312     // obrotu - obrot pozostaje "czysty" (sam balans + składowa różnicowa), co jest
313     // maksymalnie odporne na wywrocenie. Obrot i tak jest praktycznie w miejscu, bo faza
314     // przygotowania uruchamia go dopiero, gdy robot stoi (pelzanie znikome).
315     bool posHoldActive = ENABLE_POS_HOLD;
316     static bool prevPosHold = false;
317     if (posHoldActive && !prevPosHold) posEst = 0.0f; // złap bieżącą pozycję na starcie trzymania
318     prevPosHold = posHoldActive;
319
320     float setpoint = TARGET_ANGLE + angleTrim; // baza + powolna korekta (auto-trim)
321     if (posHoldActive) {
322         float dcmd = (turning || turnArmed) ? 0.0f : driveCmd; // w obrocie zawsze trzymaj pozycję
323         float vel = cmdSpeed * SPEED_TO_RATE - dcmd; // odchyłka prędkości od zadanej
324         posEst += vel * dt; // całka -> błąd pozycji [kroki]
325         float spOff = -(KPOS_P * posEst + KPOS_D * vel); // pochyl, by wrócić do pozycji/prędkości
326         spOff = constrain(spOff, -SETPOINT_OFFSET_LIMIT, SETPOINT_OFFSET_LIMIT);
327         setpoint += spOff; // jazda/utrzymanie = celowe pochylenie
328     }
329     float error = setpoint - compAngle;
330
331     // --- Bezpieczeństwo: upadek ---
332     if (fabs(error) > FALL_LIMIT) {
333         motorsEnabled = false;
334         errorIntegral = 0.0f;
335         cmdSpeed = 0.0f;
336         turning = false;
337         turnArmed = false;
338         turnCmd = 0.0f;
339         posEst = 0.0f;
340         setMotorRates(0, 0);
341         lastTurnMs = millis(); // odlicz obrot od nowa po podniesieniu
342         return;
343     }
344     motorsEnabled = true;
345
346     // --- PID ---
347     errorIntegral += error * dt;
348     errorIntegral = constrain(errorIntegral, -I_LIMIT, I_LIMIT);
349     float dRaw = -gyroX;
350     dTermFilt += D_LPF_ALPHA * (dRaw - dTermFilt);
351     float output = Kp * error + Ki * errorIntegral + Kd * dTermFilt;
352
353     // --- Wygładzenie komendy prędkości (balans) ---
354     output = constrain(output, -MAX_SPEED, MAX_SPEED);
355     float maxDelta = MAX_ACCEL * dt;
356     float delta = constrain(output - cmdSpeed, -maxDelta, maxDelta);

```

```

357 cmdSpeed += delta;
358
359 // --- AUTO-TRIM: powolne dostrajanie punktu rownowagi ---
360 // Jezeli robot ciagle jedzie w jedna strone (cmdSpeed != 0), znaczy ze setpoint jest
361 // lekko przesuniety -> powoli koryguj, az srednia predkosc kol bedzie ~0 (robot stoi).
362 // Wstrzymane podczas obrotu/jazdy, by ich nie "rozregulowac".
363 #if AUTO_TRIM
364   if (!turning && !turnArmed) {
365     angleTrim += AUTO_TRIM_GAIN * cmdSpeed * dt;
366     angleTrim = constrain(angleTrim, -TRIM_LIMIT, TRIM_LIMIT);
367   }
368 #endif
369
370 // --- Harmonogram obrotu: UZBROJENIE -> PRZYGOTOWANIE -> OBROT ---
371 uint32_t nowMs = millis();
372 // 1) co TURN_INTERVAL_MS uzbrajamy obrot (jeszcze nie startujemy)
373 if (!turning && !turnArmed && (nowMs - lastTurnMs) >= TURN_INTERVAL_MS) {
374   turnArmed = true; settleTimer = 0.0f; armMs = nowMs;
375 }
376 // 2) PRZYGOTOWANIE: obrot startuje dopiero, gdy robot jest pionowo i spokojny
377 // przez TURN_READY_TIME (lub po przekroczeniu TURN_ARM_TIMEOUT_MS - obrot mimo to)
378 if (turnArmed) {
379   bool calm = fabs(error) < TURN_READY_ANGLE
380     && fabs(gyroX) < TURN_READY_RATE
381     && fabs(cmdSpeed) < TURN_READY_SPEED;
382   settleTimer = calm ? (settleTimer + dt) : 0.0f;
383   if (settleTimer >= TURN_READY_TIME || (nowMs - armMs) >= TURN_ARM_TIMEOUT_MS) {
384     turnArmed = false; turning = true; turnStepsLeft = TURN_STEPS;
385   }
386 }
387 // --- Predkosc roznicowa obrotu z PROPORCJONALNYM priorytetem balansu ---
388 float turnTarget = 0.0f;
389 if (turning) {
390   turnStepsLeft -= fabs(turnCmd) * dt; // licz wykonany luk wg AKTUALNEJ predkosci
391   if (turnStepsLeft <= 0.0f) { turning = false; lastTurnMs = nowMs; }
392   else {
393     // wspolczynnik 1 (pion) -> 0 (przechyl TURN_PITCH_STOP): obrot zwalnia, nie blokuje sie
394     float m = (TURN_PITCH_STOP - fabs(error)) / (TURN_PITCH_STOP - TURN_PITCH_FULL);
395     m = constrain(m, 0.0f, 1.0f);
396     turnTarget = TURN_DIR * TURN_RATE * m;
397   }
398 }
399 if (turning && fabs(error) >= TURN_PITCH_EMERG) {
400   turnCmd = 0.0f; // awaryjne natychmiastowe odcięcie (duzy przechyl)
401 } else {
402   float dTurn = TURN_RAMP * dt; // plynna rampa
403   turnCmd += constrain(turnTarget - turnCmd, -dTurn, dTurn);
404 }
405
406 // --- Opcjonalne utrzymanie kursu (yaw) - rozna predkosc kol ---
407 float yawCorr = 0.0f;
408 #if ENABLE_YAW_HOLD
409   if (!turning && fabs(turnCmd) < 50.0f) { // nie koliduj z zamierzonym obrotem
410     heading += yawRate * dt; // scalony kurs
411     yawCorr = -(KYAW_P * heading + KYAW_D * yawRate);
412     yawCorr = constrain(yawCorr, -YAW_CORR_LIMIT, YAW_CORR_LIMIT);
413   } else {
414     heading = 0.0f; // po obrocie trzymaj NOWY kurs
415   }
416 #endif
417
418 // --- Skladanie: balans (wspolny) +/- (obrot + korekta kursu), do silnikow ---
419 float rateBalance = cmdSpeed * SPEED_TO_RATE; // mikrokroki/s
420 float diff = turnCmd + yawCorr; // skladowa roznicowa
421 setMotorRates(rateBalance - diff, rateBalance + diff);
422
423 // --- Debug (opcjonalny) ---
424 #if DEBUG_SERIAL
425   if (++dbgCounter >= DEBUG_EVERY_N) {
426     dbgCounter = 0;
427     Serial.print("ang="); Serial.print(compAngle, 2);
428     Serial.print(" tgt="); Serial.print(TARGET_ANGLE, 2); // przechwycony kat rownowagi
429     Serial.print(" trim="); Serial.print(angleTrim, 2); // auto-trim

```

```
430     Serial.print(" err="); Serial.print(error, 2);
431     Serial.print(" out="); Serial.print(output, 0);
432     Serial.print(" st="); Serial.print(turning ? 2 : (turnArmed ? 1 : 0)); // 0=balans 1=przygot. 2=obrot
433     Serial.print(" tc="); Serial.println(turnCmd, 0); // biezaca predkosc obrotu
434 }
435 #endif
436 }
```